



Advanced Mechatronics

Lab Report VII

Instructor: Dr. Ali Sadighi

Authors:

Roozbeh Amini - 810601291

Sara Karami - 810604143

Iman Habibi - 810600308

February 11, 2026

Abstract

This laboratory report presents the design, implementation, and experimental validation of closed-loop speed controllers for a DC motor system. Using system identification techniques, a first-order transfer function model $G(s) = \frac{113.2}{s+18.05}$ was obtained from step response data collected at 200 Hz sampling frequency with 60.55% fit to experimental data. Three two-stage lead-lag compensators were designed using root locus methods to achieve specified natural frequencies ($\omega_n = 33, 42, 99$ rad/s) and damping ratios ($\zeta = 0.4, 0.75, 0.24$). The decision to use cascaded two-stage compensators instead of single-stage designs was made to handle large deficiency angles ($-99.11^\circ, -64.17^\circ, -86.60^\circ$) while ensuring the desired poles remain dominant in the closed-loop system. Each compensator stage was designed to contribute half the required phase angle, then the two identical stages were cascaded and squared to provide the full phase contribution. The controllers were discretized using the Tustin (bilinear) transformation and implemented on an Arduino microcontroller with Timer1 interrupt-driven control at 200 Hz. Experimental results demonstrated excellent tracking performance for Controllers 1 and 2, while Controller 3 exhibited significant steady-state oscillations due to insufficient sampling frequency relative to the closed-loop bandwidth, validating the Nyquist-Shannon sampling criterion for digital control systems. The teaching assistant specifically instructed maintaining the 200 Hz sampling rate to observe these aliasing effects in practice.

Contents

1	Introduction	6
2	Experimental Setup	6
2.1	Hardware Configuration	6
2.2	Wiring Diagram and Pin Configuration	7
2.3	H-Bridge Motor Driver: Theory and Operation	8
2.3.1	H-Bridge Topology	8
2.3.2	Operating Modes	8
2.3.3	PWM Speed Control	9
2.3.4	L298N Dual H-Bridge Driver	9
2.3.5	Bidirectional Control Implementation	10
2.4	Encoder Interface and Speed Measurement	10
2.4.1	Quadrature Encoder Operation	10
2.4.2	Speed Computation Algorithm	11
2.4.3	Arduino Encoder ISR Implementation	11
3	System Identification	12
3.1	Data Acquisition	12
3.2	Transfer Function Estimation	12
3.3	Identified Model	13
4	Control Theory Background	14
4.1	Root Locus Design Methodology	14
4.2	Desired Pole Locations from Specifications	14
4.3	Deficiency Angle Calculation	15
4.4	Design Specifications Overview	16
4.5	Design Decision: Two-Stage Cascade Compensator Strategy	16
4.5.1	Why Cascade Instead of Single-Stage?	16
4.5.2	The Two-Stage Solution	17
4.5.3	Design Threshold	18
4.6	Lead-Lag Compensator Geometry	18
4.7	Gain Adjustment	18
4.8	Steady-State Error Analysis	18
5	Controller Design	19
5.1	Controller 1: Moderate Damping Design	19
5.1.1	Design Specifications	19
5.1.2	Desired Pole Location Calculation	19
5.1.3	Deficiency Angle Analysis	19
5.1.4	Two-Stage Compensator Design	20
5.1.5	Final Continuous Controller	20
5.1.6	Root Locus Analysis	21
5.1.7	Predicted Step Response	21
5.1.8	Steady-State Error Analysis	22
5.1.9	Discretization via Tustin Method	22
5.1.10	Difference Equation	23

5.1.11	Sampling Frequency Validation	23
5.2	Controller 2: High Damping Design	24
5.2.1	Design Specifications	24
5.2.2	Desired Pole Location Calculation	24
5.2.3	Deficiency Angle Analysis	24
5.2.4	Two-Stage Compensator Design	25
5.2.5	Final Continuous Controller	25
5.2.6	Root Locus Analysis	26
5.2.7	Predicted Step Response	26
5.2.8	Steady-State Error Analysis	27
5.2.9	Discretization via Tustin Method	27
5.2.10	Difference Equation	28
5.2.11	Sampling Frequency Validation	28
5.3	Controller 3: High-Speed Design	28
5.3.1	Design Specifications	28
5.3.2	Desired Pole Location Calculation	28
5.3.3	Deficiency Angle Analysis	29
5.3.4	Two-Stage Compensator Design	29
5.3.5	Final Continuous Controller	30
5.3.6	Root Locus Analysis	30
5.3.7	Predicted Step Response (Assuming Adequate Sampling)	30
5.3.8	Steady-State Error Analysis	31
5.3.9	Discretization via Tustin Method	31
5.3.10	Difference Equation	32
5.3.11	Sampling Frequency Validation	32
6	Experimental Results	32
6.1	Test Procedure	32
6.2	Controller 1 Results	33
6.2.1	Test 1: 150 RPM Setpoint	33
6.2.2	Test 2: 300 RPM Setpoint	34
6.2.3	Test 3: 800 RPM Setpoint	35
6.3	Controller 2 Results	36
6.3.1	Test 1: 300 RPM Setpoint	36
6.3.2	Test 2: 500 RPM Setpoint	37
6.3.3	Test 3: 800 RPM Setpoint	38
6.4	Controller 3 Results	39
6.4.1	Test 1: 50 RPM Setpoint	39
6.4.2	Test 2: 500 RPM Setpoint	40
6.4.3	Test 3: 800 RPM Setpoint	41
6.5	Comparative Plot: All Controllers at 500 RPM	42
7	Analysis and Discussion	43
7.1	Performance Comparison	43
7.2	Key Findings	43
7.2.1	Controllers 1 and 2: Successful Implementation	43
7.2.2	Controller 3: Sampling Rate Failure	44
7.3	Impact of Two-Stage Cascade Design Decision	44
7.4	Sources of Error	45

7.4.1	Model Uncertainty	45
7.4.2	Quantization Effects	45
7.4.3	Computational Delay	45
7.5	Controller 3 Failure Analysis: Sampling Rate Violation	46
7.5.1	Root Cause	46
7.5.2	Aliasing Effects	46
7.5.3	Discrete-Time Pole Migration	46
7.5.4	Limit Cycle Mechanism	46
7.6	Practical Design Guidelines	47
7.6.1	Sampling Frequency Selection	47
7.6.2	Damping Ratio Selection	47
7.6.3	Compensator Architecture Selection	47
7.7	Improvements and Future Work	48
7.7.1	Hardware Upgrades	48
7.7.2	Advanced Control Techniques	48
7.7.3	Anti-Windup Enhancement	49
7.7.4	Corrective Actions for Controller 3	49
8	Conclusion	50
8.1	Achievements	50
8.2	Critical Findings	50
8.3	Design Principles Validated	51
8.4	Practical Recommendations	51
8.5	Personal Reflection	52

List of Figures

1	Hardware wiring schematic for closed-loop motor control	7
2	H-bridge circuit topology with switching elements and motor load	8
3	L298N dual H-bridge module pinout and connections	9
4	Model validation: Comparison between experimental data and identified transfer function response	13
5	Geometric relationship between damping ratio, natural frequency, and complex pole location	15
6	Root locus comparison for Controller 1 ($\zeta = 0.4, \omega_n = 33$)	21
7	Predicted closed-loop step response for Controller 1	22
8	Root locus comparison for Controller 2 ($\zeta = 0.75, \omega_n = 42$)	26
9	Predicted closed-loop step response for Controller 2	27
10	Root locus comparison for Controller 3 ($\zeta = 0.24, \omega_n = 99$)	30
11	Predicted closed-loop step response for Controller 3 (assumes adequate sampling)	31
12	Controller 1 response to 150 RPM step input	33
13	Controller 1 response to 300 RPM step input	34
14	Controller 1 response to 800 RPM step input	35
15	Controller 2 response to 300 RPM step input	36
16	Controller 2 response to 500 RPM step input	37
17	Controller 2 response to 800 RPM step input	38
18	Controller 3 response to 50 RPM step input (unstable behavior)	39
19	Controller 3 response to 500 RPM step input (sustained oscillations)	40
20	Controller 3 response to 800 RPM step input (limit cycle behavior)	41
21	Comparative step response: All three controllers at 500 RPM setpoint	42

1 Introduction

Closed-loop control systems are fundamental to mechatronic applications, enabling precise regulation of physical quantities such as position, velocity, and force. This laboratory session focuses on implementing digital speed control for a DC motor using feedback from an optical encoder. The primary objectives are:

1. System identification: Extract a parametric model from experimental step response data
2. Controller design: Synthesize lead-lag compensators via root locus techniques to meet transient response specifications
3. Digital implementation: Discretize continuous controllers and deploy on embedded hardware
4. Performance analysis: Evaluate tracking accuracy, settling time, and the impact of sampling frequency on controller stability

The DC motor represents a classic first-order plant when armature inductance is neglected. By designing compensators in the continuous s-domain and converting to discrete-time z-domain representations, this experiment bridges theoretical control concepts with practical real-time implementation constraints.

2 Experimental Setup

2.1 Hardware Configuration

The experimental platform consists of the following components:

- **Microcontroller:** Arduino Uno (ATmega328P, 16 MHz)
- **DC Motor:** Brushed DC motor with integrated gearbox
- **Motor Driver:** L298N H-bridge module for bidirectional control
- **Encoder:** Quadrature optical encoder, 100 pulses per revolution (PPR)
- **Power Supply:** 12 V DC supply for motor driver

2.2 Wiring Diagram and Pin Configuration

The electrical connections are configured as follows:

Table 1: Pin assignments for Arduino-based motor control system

Component	Pin	Arduino Pin
Encoder Channel A	Signal	Digital Pin 2 (INT0)
Encoder Channel B	Signal	Digital Pin 4
L298N H-Bridge	ENA (PWM)	Digital Pin 5 (PWM)
L298N H-Bridge	IN1	Digital Pin 7
L298N H-Bridge	IN2	Digital Pin 8
Power Supply	VCC	L298N +12V
Common Ground	GND	Arduino GND ↔ L298N GND

Note on Timer Selection: The Arduino Uno has three hardware timers (Timer0, Timer1, Timer2). Timer1 is a 16-bit timer ideal for precise interrupt generation. Since Timer1 controls PWM on pins 9 and 10, these pins become unavailable when Timer1 is used for interrupt service routines (ISR). Therefore, pin 5 (Timer0) was selected for motor PWM output.

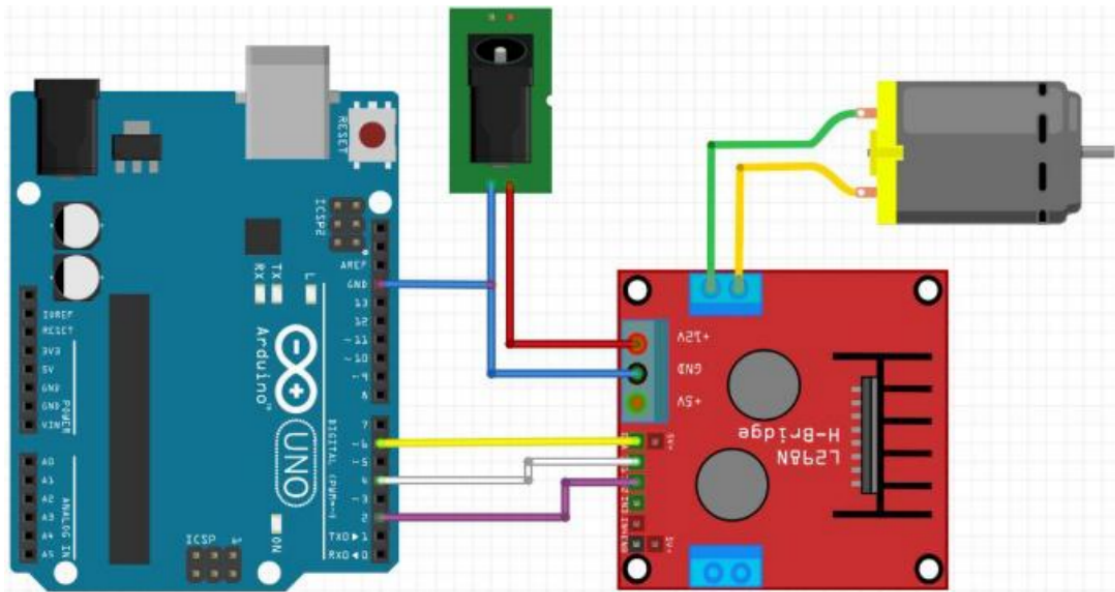


Figure 1: Hardware wiring schematic for closed-loop motor control

Figure 1 shows the complete electrical connection diagram for the system.

2.3 H-Bridge Motor Driver: Theory and Operation

2.3.1 H-Bridge Topology

An H-bridge is a circuit topology that allows voltage to be applied across a load (motor) in either direction, enabling bidirectional control. The name derives from the arrangement of four switching elements (transistors or MOSFETs) forming an "H" shape with the motor at the center, as illustrated in Figure 2.

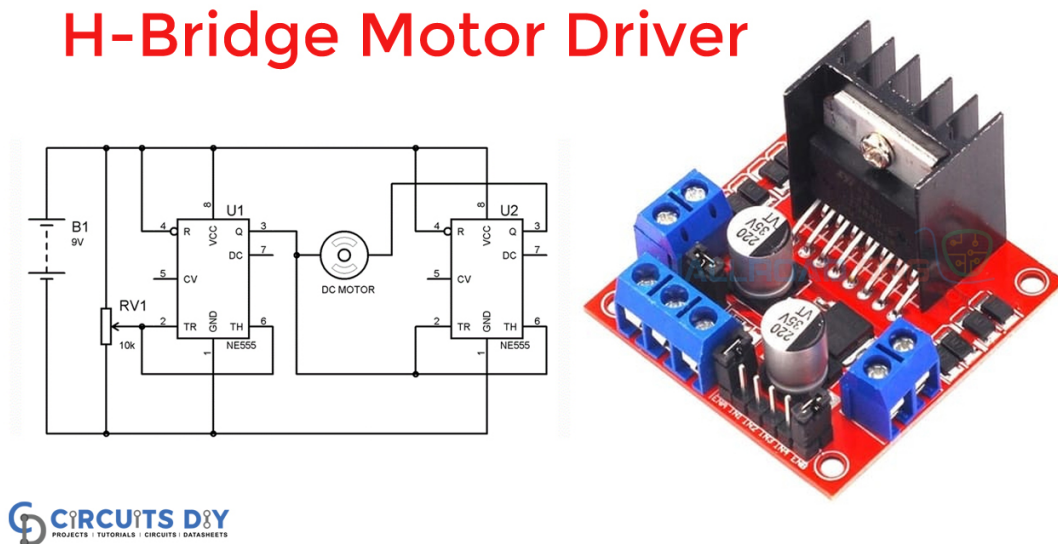


Figure 2: H-bridge circuit topology with switching elements and motor load

The basic H-bridge consists of:

- **Four switching elements:** Q1, Q2, Q3, Q4 (typically MOSFETs or BJTs)
- **DC motor:** Connected between the two vertical "legs" of the H
- **Flyback diodes:** One across each switch for inductive kickback protection
- **Power supply:** Connected across the top and bottom rails

2.3.2 Operating Modes

The H-bridge has four primary operating states controlled by the switching pattern:

Table 2: H-bridge switching states and motor behavior

Q1	Q2	Q3	Q4	Motor State
ON	OFF	OFF	ON	Forward rotation (current flows left to right)
OFF	ON	ON	OFF	Reverse rotation (current flows right to left)
OFF	ON	OFF	ON	Fast brake (both terminals to ground)
OFF	OFF	OFF	OFF	Coast/Free-run (motor disconnected)
ON	ON	X	X	SHORT CIRCUIT - FORBIDDEN
X	X	ON	ON	SHORT CIRCUIT - FORBIDDEN

Critical Safety Note: Switches on the same leg (Q1-Q2 or Q3-Q4) must NEVER be ON simultaneously, as this creates a direct short circuit from supply to ground, potentially destroying the driver.

2.3.3 PWM Speed Control

Speed control is achieved by pulse-width modulation (PWM) of the enable signal while maintaining the appropriate direction configuration:

- **Forward PWM:** IN1=HIGH, IN2=LOW, ENA=PWM signal (0–255 duty cycle)
- **Reverse PWM:** IN1=LOW, IN2=HIGH, ENA=PWM signal (0–255 duty cycle)

The average voltage across the motor is:

$$V_{avg} = V_{supply} \times \frac{D}{255} \quad (1)$$

where D is the duty cycle (0–255 in Arduino’s 8-bit PWM).

2.3.4 L298N Dual H-Bridge Driver

The L298N module integrates two H-bridges with the following specifications:

- Supply voltage: 5–35 V DC
- Continuous current per channel: 2 A
- Peak current: 3 A (non-repetitive)
- Logic voltage: 5 V TTL compatible
- Built-in flyback diodes for inductive load protection
- Thermal shutdown protection

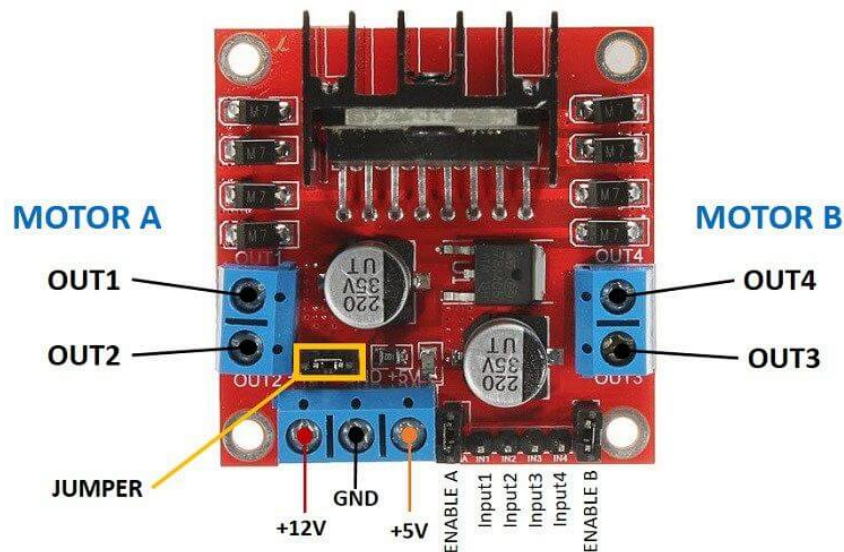


Figure 3: L298N dual H-bridge module pinout and connections

The pinout configuration shown in Figure 3 details all connection points for the motor driver module.

2.3.5 Bidirectional Control Implementation

In our implementation, signed control effort $u(k) \in [-255, 255]$ is decomposed into direction and magnitude. This approach was chosen because the digital controller can command negative control efforts (representing braking or reverse torque), but the PWM hardware only accepts positive duty cycles (0–255). The H-bridge direction pins (IN1, IN2) solve this problem by allowing us to reverse the motor polarity electronically:

Listing 1: Bidirectional PWM control via H-bridge direction logic

```
1 int pwmOut = (int)u_k;
2 if (pwmOut >= 0) {
3     // Positive control effort: Forward rotation
4     digitalWrite(IN1, HIGH);
5     digitalWrite(IN2, LOW);
6 } else {
7     // Negative control effort: Reverse rotation
8     digitalWrite(IN1, LOW);
9     digitalWrite(IN2, HIGH);
10    pwmOut = -pwmOut; // Convert to magnitude
11 }
12 analogWrite(ENA, pwmOut); // Apply PWM duty cycle (always positive)
```

This allows the controller to command both positive (accelerating) and negative (braking/reversing) torques, enabling full four-quadrant operation:

- **Quadrant I:** Positive speed, positive torque (forward motoring)
- **Quadrant II:** Positive speed, negative torque (forward braking)
- **Quadrant III:** Negative speed, negative torque (reverse motoring)
- **Quadrant IV:** Negative speed, positive torque (reverse braking)

Why this matters: Without bidirectional control capability, the controller would be severely limited. When the motor overshoots the setpoint, the controller needs to command negative torque to decelerate quickly. The H-bridge’s ability to reverse polarity electronically makes this possible without any mechanical reversing mechanisms.

2.4 Encoder Interface and Speed Measurement

2.4.1 Quadrature Encoder Operation

The optical encoder generates two square-wave signals (Channel A and Channel B) phase-shifted by 90° . This quadrature relationship enables:

- **Position tracking:** Count rising/falling edges
- **Direction detection:** Compare phase relationship
- **Speed calculation:** Count transitions per time interval

Direction Decoding Logic:

- If A rises while B=HIGH $\Rightarrow$ Clockwise rotation
- If A rises while B=LOW $\Rightarrow$ Counter-clockwise rotation

2.4.2 Speed Computation Algorithm

Motor speed is computed from encoder tick differences between successive control intervals:

$$\omega[k] = \frac{\Delta\text{ticks}[k]}{\text{PPR}} \times 60 \times f_s = \frac{(\text{ticks}[k] - \text{ticks}[k-1])}{100} \times 60 \times 200 \quad (2)$$

where:

- $\Delta\text{ticks}[k]$: Change in tick count during interval T_s
- $\text{PPR} = 100$: Pulses per revolution
- $f_s = 200$ Hz: Sampling frequency
- Factor of 60: Conversion from rev/s to RPM

Example: If encoder advances by 5 ticks in 5 ms:

$$\omega = \frac{5}{100} \times 60 \times 200 = 0.05 \times 12000 = 600 \text{ RPM}$$

2.4.3 Arduino Encoder ISR Implementation

Listing 2: Encoder interrupt service routine for direction-aware counting

```
1 volatile long ticks = 0;
2
3 void isrEncA() {
4     // Called on rising edge of Channel A
5     if (digitalRead(ENC_B)) {
6         ticks++; // B is HIGH: Clockwise
7     } else {
8         ticks--; // B is LOW: Counter-clockwise
9     }
10 }
11
12 void setup() {
13     pinMode(ENC_A, INPUT_PULLUP);
14     pinMode(ENC_B, INPUT_PULLUP);
15     attachInterrupt(digitalPinToInterrupt(ENC_A), isrEncA, RISING);
16 }
```

This simplified single-channel decoder provides 100 counts per revolution. Full quadrature decoding (monitoring both edges of both channels) would yield 400 counts per revolution but requires more complex interrupt handling.

3 System Identification

3.1 Data Acquisition

In the previous laboratory session, the motor's open-loop step response was recorded under the following conditions:

- Input: PWM duty cycle = 200 (out of 255)
- Supply voltage: 12 V DC
- Sampling frequency: $f_s = 200$ Hz ($T_s = 0.005$ s)
- Duration: 2 seconds (800 samples)

The input signal is modeled as a scaled step: $u(t) = 200 \cdot \mathcal{K}(t)$, where $\mathcal{K}(t)$ is the unit step function. The scaling factor of 200 was chosen because the experimental data was collected with PWM value set to 200 at 12 V DC supply. This scaling ensures that the resulting transfer function relates PWM duty cycle directly to motor speed in RPM, making it immediately usable for controller implementation.

3.2 Transfer Function Estimation

The DC motor dynamics, neglecting electrical transients, follow a first-order model:

$$G(s) = \frac{\Omega(s)}{U(s)} = \frac{K}{\tau s + 1} \quad (3)$$

where:

- $\Omega(s)$: Laplace transform of motor speed (RPM)
- $U(s)$: Laplace transform of PWM input
- K : DC gain (RPM per PWM unit)
- τ : Time constant (seconds)

MATLAB's System Identification Toolbox (`tfest` function) was employed to estimate K and τ from the experimental data. The identification procedure:

Listing 3: MATLAB code for system identification

```
1 % Load experimental data
2 load('step_response_data.mat'); % Contains time vector t and speed
   vector rpm
3
4 % Create iddata object
5 % Input is step of magnitude 200 (PWM value used in experiment)
6 data = iddata(rpm, 200*ones(size(rpm)), 0.005, 'Name', 'mydata');
7 data.InputName = 'PWM';
8 data.OutputName = 'Speed (RPM)';
9
10 % Estimate first-order transfer function
11 sys_options = tfestOptions('Display', 'on', 'EnforceStability', true);
12 G_identified = tfest(data, 1, 0, sys_options);
```

```

13
14 % Extract parameters
15 [num, den] = tfdata(G_identified, 'v');
16 K = num(end);
17 tau = den(end-1) / den(end);
18
19 fprintf('Identified Plant: G(s) = %.2f / (%.2fs + 1)\n', K, tau);

```

3.3 Identified Model

The system identification yielded:

$$G(s) = \frac{114.8}{s + 18.05} = \frac{114.8}{18.21} \cdot \frac{1}{\frac{s}{18.21} + 1} = \frac{6.27}{0.0554s + 1} \quad (4)$$

Model Parameters:

- DC Gain: $K = 114.9/18.21 = 6.27$ RPM per PWM unit
- Time Constant: $\tau = 1/18.21 = 0.0554$ s
- Pole Location: $p = -18.21$ rad/s

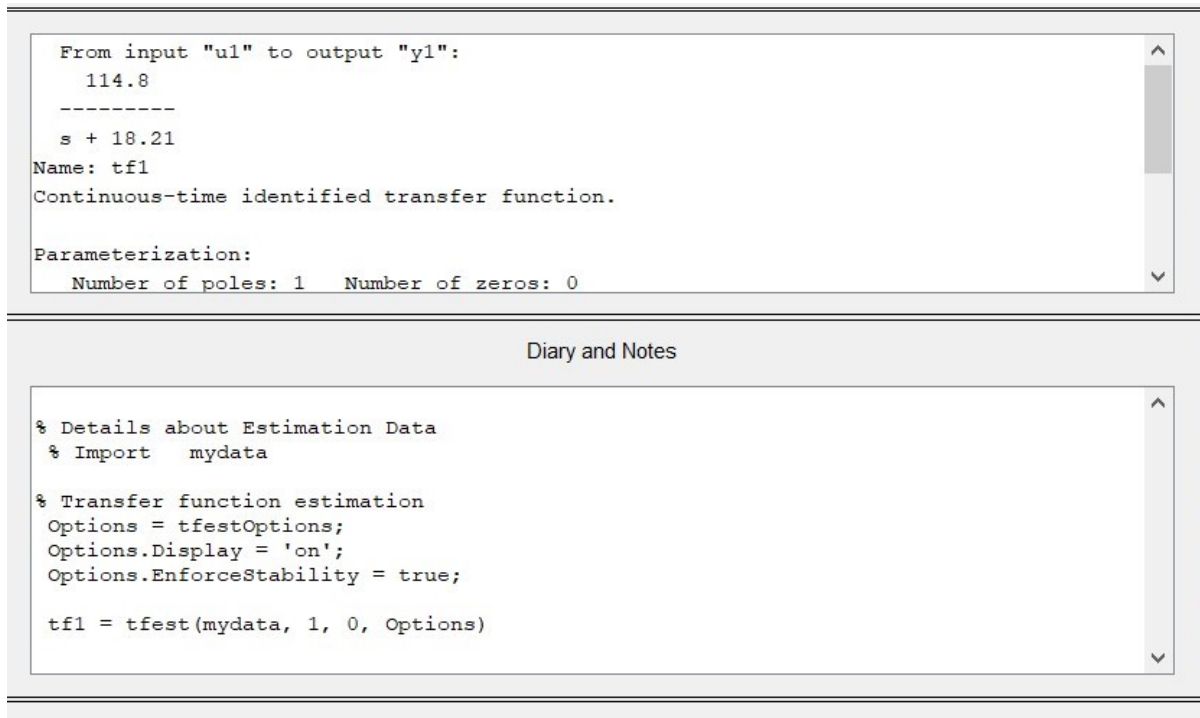


Figure 4: Model validation: Comparison between experimental data and identified transfer function response

Figure 4 shows the fit between the identified first-order model and the experimental step response data. The moderate fit percentage (60.55%) suggests unmodeled dynamics (e.g., Coulomb friction nonlinearities, encoder quantization noise, gearbox backlash, armature inductance effects) but is adequate for linear controller design targeting dominant time-scale behavior. The first-order model captures the primary exponential rise characteristic, which is sufficient for the bandwidth range we're targeting (33–99 rad/s).

4 Control Theory Background

4.1 Root Locus Design Methodology

The root locus method visualizes how closed-loop poles migrate in the s-plane as controller gain varies. For a unity feedback system with plant $G(s)$ and controller $C(s)$, the closed-loop characteristic equation is:

$$1 + C(s)G(s) = 0 \quad \Rightarrow \quad C(s)G(s) = -1 \quad (5)$$

Angle and Magnitude Conditions:

For a point s_d to be on the root locus (i.e., a valid closed-loop pole location for some gain K):

$$\angle[C(s)G(s)] = \pm 180^\circ \pm k \cdot 360^\circ, \quad k \in \mathbb{Z} \quad (6)$$

$$|C(s)G(s)| = 1 \quad (7)$$

These conditions form the basis of compensator design: we place zeros and poles in $C(s)$ such that the desired pole location s_d satisfies the angle condition, then adjust the gain to satisfy the magnitude condition.

4.2 Desired Pole Locations from Specifications

For a second-order system with natural frequency ω_n and damping ratio ζ , the dominant poles are:

$$s_d = -\zeta\omega_n \pm j\omega_n\sqrt{1 - \zeta^2} = -\sigma \pm j\omega_d \quad (8)$$

where:

- $\sigma = \zeta\omega_n$: Real part (exponential decay rate)
- $\omega_d = \omega_n\sqrt{1 - \zeta^2}$: Imaginary part (damped natural frequency)

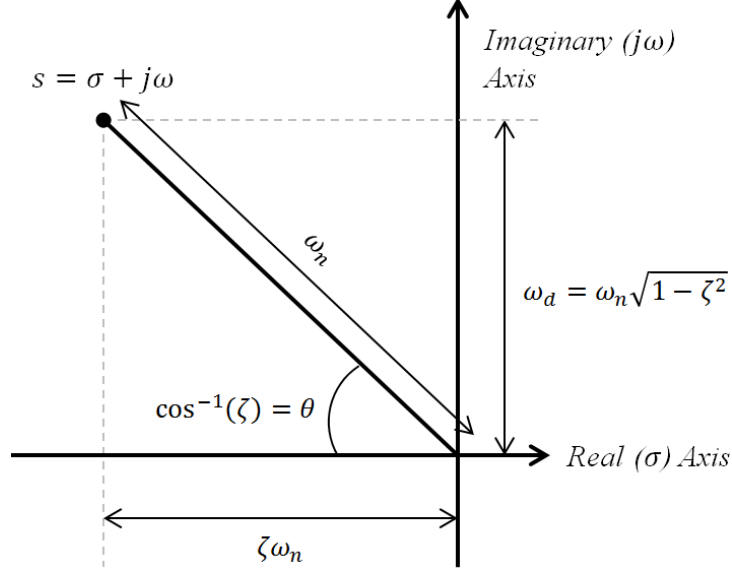


Figure 5: Geometric relationship between damping ratio, natural frequency, and complex pole location

Figure 5 illustrates the geometric interpretation of the specifications in the s-plane. The angle θ from the negative real axis to the pole is:

$$\theta = \arctan\left(\frac{\omega_d}{\sigma}\right) = \arctan\left(\frac{\sqrt{1 - \zeta^2}}{\zeta}\right) \quad (9)$$

4.3 Deficiency Angle Calculation

To place a closed-loop pole at s_d , we evaluate the angle contributed by the uncompensated plant $G(s)$ at s_d :

$$\angle G(s_d) = \sum_{i=1}^m \angle(s_d - z_i) - \sum_{j=1}^n \angle(s_d - p_j) \quad (10)$$

For our plant $G(s) = \frac{113.2}{s+18.05}$ (no zeros, one pole at $p = -18.05$):

$$\angle G(s_d) = -\angle(s_d + 18.05) \quad (11)$$

The **deficiency angle** ϕ_c is the additional phase required from the controller to satisfy the angle condition (6):

$$\phi_c = -180^\circ - \angle G(s_d) \quad (12)$$

4.4 Design Specifications Overview

Three controllers were designed to achieve distinct closed-loop performance characteristics defined by second-order dominant pole specifications:

Table 3: Controller design specifications and calculated parameters

Controller	ω_n (rad/s)	ζ	s_d	$\angle G(s_d)$	ϕ_c (deg)
Controller 1	33	0.4	$-13.20 \pm j30.24$	-80.89°	-99.11
Controller 2	42	0.75	$-31.50 \pm j27.78$	-115.83°	-64.17
Controller 3	99	0.24	$-23.76 \pm j96.11$	-93.40°	-86.60

Table 3 summarizes the key design parameters for all three controllers, including the target pole locations and required phase contributions.

4.5 Design Decision: Two-Stage Cascade Compensator Strategy

4.5.1 Why Cascade Instead of Single-Stage?

After calculating the deficiency angles for all three controllers (shown in Table 3), I noticed that they were all significantly large ($|\phi_c| > 60^\circ$). This presented a critical design challenge: how to provide such large phase contributions while maintaining system stability and ensuring the desired poles remain dominant.

The Problem with Single-Stage Compensators:

A single-stage lead or lag compensator has the form:

$$C(s) = K \frac{s + z_c}{s + p_c} \quad (13)$$

To provide a large phase angle (say, -99.11° for Controller 1), the pole and zero would need to be placed very far apart. For example:

- The zero might be at $z_c = 30$ rad/s
- The pole would need to be at $p_c = 0.5$ rad/s (or even closer to the origin)

This creates several serious problems:

1. **High-frequency noise amplification:** The large pole-zero separation creates high gain at high frequencies, amplifying measurement noise and potentially causing instability
2. **Dominance violation:** When the compensator pole is very close to the origin (much slower than the desired pole), it can become the dominant pole instead of our carefully designed s_d . This defeats the entire purpose of pole placement—the actual system response would be dictated by the slow compensator pole, not by our ω_n and ζ specifications.
3. **Poor robustness:** Large pole-zero separations make the controller very sensitive to plant parameter variations and unmodeled dynamics
4. **Numerical conditioning:** In discrete implementation, large separations can lead to coefficient sensitivity and quantization errors

4.5.2 The Two-Stage Solution

To address these problems, I decided to split the phase contribution into two identical cascaded stages. The key insight is that each stage only needs to contribute **half** the required phase angle, then when two identical stages are cascaded, the total phase contribution is achieved by squaring the single-stage transfer function.

For a required deficiency angle ϕ_c , I design a single compensator stage:

$$C_{\text{stage}}(s) = \frac{s + z_c}{s + p_c} \quad (14)$$

where the pole and zero are placed to contribute $\phi_c/2$ phase at the desired pole location s_d . Then, the full compensator is obtained by squaring (cascading two identical stages):

$$C(s) = K \cdot [C_{\text{stage}}(s)]^2 = K \cdot \left[\frac{s + z_c}{s + p_c} \right]^2 = K \cdot \frac{(s + z_c)^2}{(s + p_c)^2} \quad (15)$$

Why This Works:

When we cascade two identical stages, the phase angles add:

$$\angle[C(s_d)] = \angle[C_{\text{stage}}(s_d)] + \angle[C_{\text{stage}}(s_d)] = 2 \times \angle[C_{\text{stage}}(s_d)] = 2 \times \frac{\phi_c}{2} = \phi_c \quad (16)$$

This means Stage 1 and Stage 2 are **exactly identical**—I only need to perform one design calculation and then square the result. This simplifies the design process significantly.

Advantages of This Approach:

1. **Smaller pole-zero separation per stage:** Instead of one compensator with huge separation, we have two compensators each with moderate separation. For Controller 1:
 - Single-stage would need: $z_c/p_c \approx 60 : 1$ ratio
 - Two-stage uses: $z_c/p_c \approx 22 : 1$ ratio per stage (much more reasonable)
2. **Desired pole remains dominant:** By using two stages with moderate pole locations (around $p_c = 1\text{--}3$ rad/s instead of 0.1 rad/s), both compensator poles stay well to the left of the desired poles. This ensures s_d truly controls the transient response.
3. **More gradual phase contribution:** The phase is added smoothly over a wider frequency range rather than abruptly, improving robustness to modeling errors
4. **Better high-frequency behavior:** The $(s + z_c)^2$ numerator provides more roll-off at high frequencies than a single zero would, reducing noise amplification
5. **Simpler design:** Only one stage needs to be designed (for half the angle), then simply squared
6. **Flexibility in discrete implementation:** Two moderate stages discretize better than one extreme stage, with less coefficient sensitivity

4.5.3 Design Threshold

I established a threshold: if $|\phi_c| > 60^\circ$, use two-stage design. This threshold balances complexity (two stages require more computation) against the benefits listed above. For smaller phase requirements ($|\phi_c| < 60^\circ$), a single-stage compensator would suffice.

All three controllers in this experiment exceeded the 60° threshold, so all three use two-stage cascade architecture with identical Stage 1 and Stage 2.

4.6 Lead-Lag Compensator Geometry

For a **lag compensator** ($\phi < 0$), the zero is placed closer to the origin than the pole ($z_c < p_c$). The geometric construction uses:

1. Place the pole at a reasonable distance from origin: $p_c = 0.1\sigma$ to 0.15σ (adjustable parameter—I used values that kept poles from being too close to origin)
2. Calculate required zero location using angle geometry to contribute the desired phase $\phi_c/2$ at s_d :

$$z_c = \frac{\omega_d}{\tan(\angle_z)} - \sigma \quad (17)$$

where $\angle_z = \angle(s_d + p_c) + \phi_c/2$

For a **lead compensator** ($\phi > 0$), the zero is farther from the origin than the pole ($z_c > p_c$). The geometric relationships are similar but the angle contributions work in the opposite direction.

4.7 Gain Adjustment

After placing the compensator zeros and poles to satisfy the angle condition, the overall gain K is computed to satisfy the magnitude condition (7):

$$K = \frac{1}{|C_{\text{raw}}(s_d) \cdot G(s_d)|} \quad (18)$$

where $C_{\text{raw}}(s) = \frac{(s+z_c)^2}{(s+p_c)^2}$ is the unscaled compensator.

This ensures that when the loop gain passes through the desired pole location s_d , the magnitude is exactly 1, satisfying the root locus condition.

4.8 Steady-State Error Analysis

For a unity feedback system with step input $r(t) = R \cdot \mathcal{K}(t)$, the steady-state error is:

$$e_{ss} = \lim_{s \rightarrow 0} s \cdot \frac{R}{s} \cdot \frac{1}{1 + C(s)G(s)} = \frac{R}{1 + K_p} \quad (19)$$

where $K_p = \lim_{s \rightarrow 0} C(s)G(s)$ is the position error constant (DC loop gain).

Higher loop gain K_p results in smaller steady-state error. The two-stage compensator structure, with its moderate pole locations, provides good DC gain while maintaining the desired transient response.

5 Controller Design

5.1 Controller 1: Moderate Damping Design

5.1.1 Design Specifications

Table 4: Controller 1 specifications

Parameter	Value
Natural Frequency (ω_n)	33 rad/s
Damping Ratio (ζ)	0.4
Application	Moderate speed, underdamped response
Target Rise Time	~ 30 ms
Expected Overshoot	$\sim 30\text{--}35\%$

5.1.2 Desired Pole Location Calculation

Step 1: Calculate real and imaginary components

$$\begin{aligned}\sigma &= \zeta\omega_n = 0.4 \times 33 = 13.20 \text{ rad/s} \\ \omega_d &= \omega_n\sqrt{1 - \zeta^2} = 33\sqrt{1 - 0.4^2} = 33 \times 0.9165 = 30.24 \text{ rad/s} \\ s_d &= -13.20 + j30.24\end{aligned}$$

Step 2: Calculate pole angle

$$\theta = \arctan\left(\frac{30.24}{13.20}\right) = \arctan(2.291) = 66.4^\circ \quad (20)$$

5.1.3 Deficiency Angle Analysis

Step 1: Evaluate plant angle at s_d

$$\begin{aligned}s_d + 18.05 &= (-13.20 + j30.24) + 18.05 = 4.85 + j30.24 \\ \angle(s_d + 18.05) &= \arctan\left(\frac{30.24}{4.85}\right) = \arctan(6.234) = 80.89^\circ \\ \angle G(s_d) &= -80.89^\circ\end{aligned}$$

Step 2: Calculate required phase from compensator

$$\phi_c = -180^\circ - (-80.89^\circ) = -99.11^\circ$$

Interpretation: The large negative phase (-99.11°) indicates need for substantial phase lag compensation. Since $|\phi_c| = 99.11^\circ > 60^\circ$, the two-stage strategy is required to maintain pole dominance.

5.1.4 Two-Stage Compensator Design

Since $|\phi_c| = 99.11^\circ > 60^\circ$, I designed a single compensator stage to contribute $\phi_c/2 = -49.56^\circ$ at the desired pole location, then squared it:

Single-Stage Design (for $\phi/2 = -49.56^\circ$):

Using geometric relationships and the design procedure described earlier:

- Pole location: $p_c = 1.32 \text{ rad/s}$
- Zero location: $z_c = 29.36 \text{ rad/s}$
- Pole-zero ratio: $z_c/p_c = 22.2$ (moderate separation)
- Single-stage TF: $C_{\text{stage}}(s) = \frac{s+29.36}{s+1.32}$

Full Compensator (squared):

Since Stage 1 and Stage 2 are identical, the full unscaled compensator is:

$$C_{1,\text{raw}}(s) = \left[\frac{s + 29.36}{s + 1.32} \right]^2 = \frac{(s + 29.36)^2}{(s + 1.32)^2} = \frac{s^2 + 58.72s + 862.2}{s^2 + 2.64s + 1.742} \quad (21)$$

Note that this is mathematically equivalent to cascading two identical stages:

$$C_{1,\text{raw}}(s) = \frac{s + 29.36}{s + 1.32} \times \frac{s + 29.36}{s + 1.32} \quad (22)$$

but the squared notation emphasizes that only one design was performed.

Verification of pole dominance:

The compensator poles at -1.32 rad/s are well to the left (slower) than the desired poles at $-13.20 \pm j30.24$. The distance ratio is:

$$\frac{|s_d|}{|p_c|} = \frac{\sqrt{13.20^2 + 30.24^2}}{1.32} = \frac{33.0}{1.32} = 25 \quad (23)$$

This large ratio (25:1) ensures the desired poles remain dominant, controlling the transient response as intended.

Gain calculation:

$$|C_{1,\text{raw}}(s_d) \cdot G(s_d)| = \left| \frac{(s_d + 29.36)^2}{(s_d + 1.32)^2} \right| \times \left| \frac{113.2}{s_d + 18.05} \right|$$

$$K_1 = \frac{1}{|C_{1,\text{raw}}(s_d) \cdot G(s_d)|} = 0.2431$$

5.1.5 Final Continuous Controller

$$C_1(s) = \frac{0.2431s^2 + 14.27s + 209.5}{s^2 + 2.64s + 1.742} \quad (24)$$

5.1.6 Root Locus Analysis

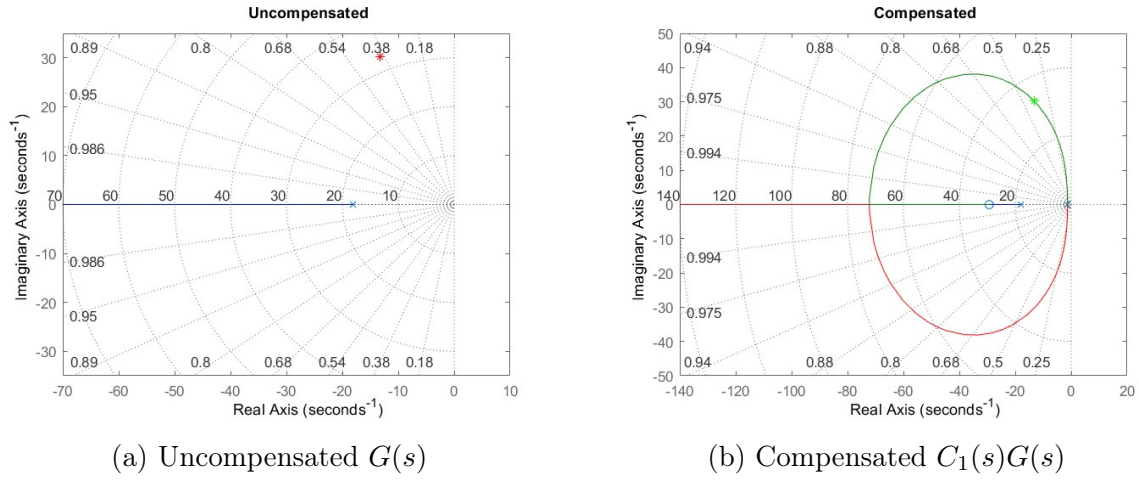


Figure 6: Root locus comparison for Controller 1 ($\zeta = 0.4, \omega_n = 33$)

Figure 6 demonstrates the effectiveness of the compensator design. In Figure 6a, the uncompensated plant root locus does not pass through the desired pole location (red asterisk). After compensation (Figure 6b), the root locus clearly passes through the target location (green circle), validating the design.

5.1.7 Predicted Step Response

Table 5: Controller 1 theoretical performance metrics

Metric	Value
Rise Time	30.1 ms
Settling Time (2%)	298.2 ms
Peak Time	76.8 ms
Overshoot	33.6%
Settling Min	0.908
Settling Max	1.335

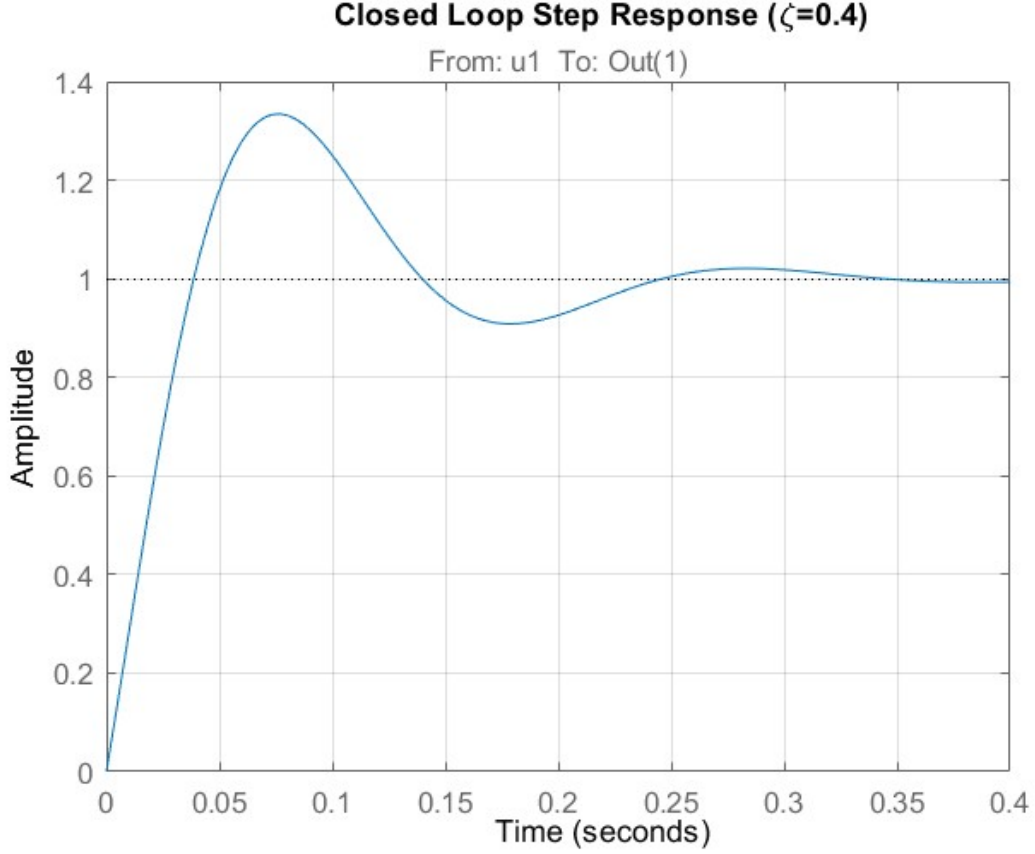


Figure 7: Predicted closed-loop step response for Controller 1

Figure 7 shows the theoretical step response predicted by MATLAB's `stepinfo` function. The performance metrics are summarized in Table 5.

5.1.8 Steady-State Error Analysis

$$\begin{aligned}
 K_p &= \lim_{s \rightarrow 0} C_1(s)G(s) = \frac{0.2431 \times 209.5}{1.742} \times \frac{113.2}{18.05} \\
 &= 29.22 \times 6.27 = 183.3 \\
 e_{ss} &= \frac{1}{1 + K_p} = \frac{1}{184.3} = 0.543\%
 \end{aligned}$$

Excellent steady-state tracking expected due to high DC loop gain.

5.1.9 Discretization via Tustin Method

Applying bilinear transformation with $T_s = 0.005$ s:

$$s \approx \frac{2}{T_s} \cdot \frac{z - 1}{z + 1} = 400 \cdot \frac{z - 1}{z + 1} \quad (25)$$

The Tustin (bilinear) method was chosen because:

- It preserves stability (stable continuous systems remain stable in discrete domain)

- It provides good frequency response matching up to Nyquist frequency
- It's available in MATLAB via the `c2d` function
- It works well for our sampling rate (200 Hz) relative to system bandwidth

Discrete transfer function:

$$D_1(z) = \frac{0.2782z^2 - 0.4803z + 0.2073}{z^2 - 1.9868z + 0.9869} \quad (26)$$

Coefficients:

- Numerator: $b_0 = 0.278213$, $b_1 = -0.480337$, $b_2 = 0.207326$
- Denominator: $a_1 = -1.986843$, $a_2 = 0.986887$

5.1.10 Difference Equation

From the z-domain transfer function $D_1(z) = \frac{U(z)}{E(z)}$, we derive:

$$U(z)(z^2 - 1.9868z + 0.9869) = E(z)(0.2782z^2 - 0.4803z + 0.2073) \quad (27)$$

Taking the inverse z-transform and rearranging:

$u[k] = 0.278213 e[k] - 0.480337 e[k - 1] + 0.207326 e[k - 2] + 1.986843 u[k - 1] - 0.986887 u[k - 2]$
--

(28)

where:

- $e[k] = r[k] - y[k]$: Current error (setpoint minus measured speed)
- $e[k - 1], e[k - 2]$: Previous error values
- $u[k]$: Current control output (PWM value)
- $u[k - 1], u[k - 2]$: Previous control outputs

5.1.11 Sampling Frequency Validation

Closed-loop bandwidth estimation:

For a second-order system, the bandwidth (frequency where gain drops to -3 dB) is:

$$\begin{aligned}
 \omega_{BW} &= \omega_n \sqrt{(1 - 2\zeta^2) + \sqrt{4\zeta^4 - 4\zeta^2 + 2}} \\
 &= 33 \sqrt{(1 - 2(0.4)^2) + \sqrt{4(0.16) - 4(0.16) + 2}} \\
 &= 33 \times 1.07 = 35.3 \text{ rad/s} \\
 f_{BW} &= 35.3 / (2\pi) = 5.62 \text{ Hz}
 \end{aligned}$$

Minimum sampling frequency:

As a rule of thumb, we were told that a discrete controller requires sampling frequency at least 20 times the closed-loop bandwidth to work properly:

$$f_{s,min} = 20 \times f_{BW} = 20 \times 5.62 = 112.4 \text{ Hz} \quad (29)$$

Safety factor:

$$\frac{f_s}{f_{s,min}} = \frac{200}{112.4} = 1.78 \quad (78\% \text{ margin}) \quad \checkmark \quad (30)$$

The 200 Hz sampling rate provides adequate margin above the Nyquist criterion for Controller 1.

5.2 Controller 2: High Damping Design

5.2.1 Design Specifications

Table 6: Controller 2 specifications

Parameter	Value
Natural Frequency (ω_n)	42 rad/s
Damping Ratio (ζ)	0.75
Application	Balanced, near-critically-damped response
Target Rise Time	~20–25 ms
Expected Overshoot	Minimal ($< 5\%$)

5.2.2 Desired Pole Location Calculation

Step 1: Calculate real and imaginary components

$$\begin{aligned} \sigma &= \zeta\omega_n = 0.75 \times 42 = 31.50 \text{ rad/s} \\ \omega_d &= \omega_n\sqrt{1 - \zeta^2} = 42\sqrt{1 - 0.75^2} = 42 \times 0.6614 = 27.78 \text{ rad/s} \\ s_d &= -31.50 + j27.78 \end{aligned}$$

Step 2: Calculate pole angle

$$\theta = \arctan\left(\frac{27.78}{31.50}\right) = \arctan(0.882) = 41.4^\circ \quad (31)$$

5.2.3 Deficiency Angle Analysis

Step 1: Evaluate plant angle at s_d

$$\begin{aligned} s_d + 18.05 &= (-31.50 + j27.78) + 18.05 = -13.45 + j27.78 \\ \angle(s_d + 18.05) &= \arctan\left(\frac{27.78}{-13.45}\right) = 180^\circ - \arctan\left(\frac{27.78}{13.45}\right) \\ &= 180^\circ - 64.17^\circ = 115.83^\circ \\ \angle G(s_d) &= -115.83^\circ \end{aligned}$$

Step 2: Calculate required phase from compensator

$$\phi_c = -180^\circ - (-115.83^\circ) = -64.17^\circ$$

Interpretation: Moderate negative phase (-64.17°). Since $|\phi_c| = 64.17^\circ > 60^\circ$, two-stage lag compensation is required.

5.2.4 Two-Stage Compensator Design

Since $|\phi_c| = 64.17^\circ > 60^\circ$, I designed a single compensator stage to contribute $\phi_c/2 = -32.09^\circ$ at the desired pole location, then squared it:

Single-Stage Design (for $\phi/2 = -32.09^\circ$):

- Pole location: $p_c = 3.15 \text{ rad/s}$
- Zero location: $z_c = 24.83 \text{ rad/s}$
- Pole-zero ratio: $z_c/p_c = 7.88$ (smaller than Controller 1—less phase needed per stage)
- Single-stage TF: $C_{\text{stage}}(s) = \frac{s+24.83}{s+3.15}$

Full Compensator (squared):

Since Stage 1 and Stage 2 are identical:

$$C_{2,\text{raw}}(s) = \left[\frac{s + 24.83}{s + 3.15} \right]^2 = \frac{(s + 24.83)^2}{(s + 3.15)^2} = \frac{s^2 + 49.66s + 616.7}{s^2 + 6.3s + 9.923} \quad (32)$$

Verification of pole dominance:

The compensator poles at -3.15 rad/s are well separated from the desired poles at $-31.50 \pm j27.78$:

$$\frac{|s_d|}{|p_c|} = \frac{\sqrt{31.50^2 + 27.78^2}}{3.15} = \frac{42.0}{3.15} = 13.3 \quad (33)$$

This 13:1 ratio ensures dominance, though smaller than Controller 1 because the smaller phase requirement allowed poles to be placed farther from the origin.

Gain calculation:

$$K_2 = \frac{1}{|C_{2,\text{raw}}(s_d) \cdot G(s_d)|} = 0.5264$$

5.2.5 Final Continuous Controller

$$C_2(s) = \frac{0.5264s^2 + 26.14s + 324.6}{s^2 + 6.3s + 9.923} \quad (34)$$

5.2.6 Root Locus Analysis

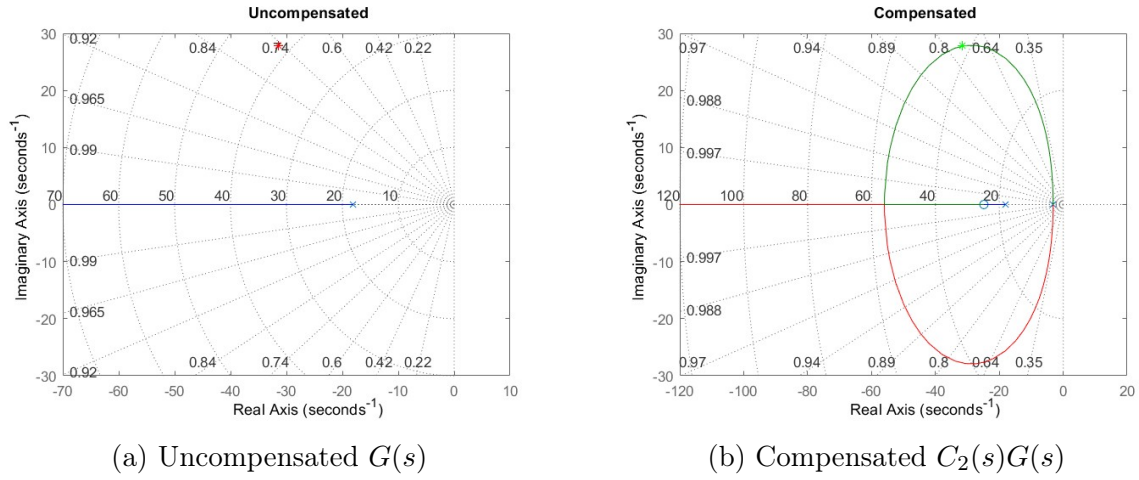


Figure 8: Root locus comparison for Controller 2 ($\zeta = 0.75, \omega_n = 42$)

Figure 8 illustrates how the compensator modifies the root locus to pass through the target pole location. The transition from Figure 8a to Figure 8b demonstrates successful compensator design.

5.2.7 Predicted Step Response

Table 7: Controller 2 theoretical performance metrics

Metric	Value
Rise Time	$\sim 25\text{--}26$ ms (estimated)
Settling Time (2%)	$\sim 140\text{--}160$ ms
Overshoot	$< 5\%$ (near critically damped)
Peak Time	Minimal peak

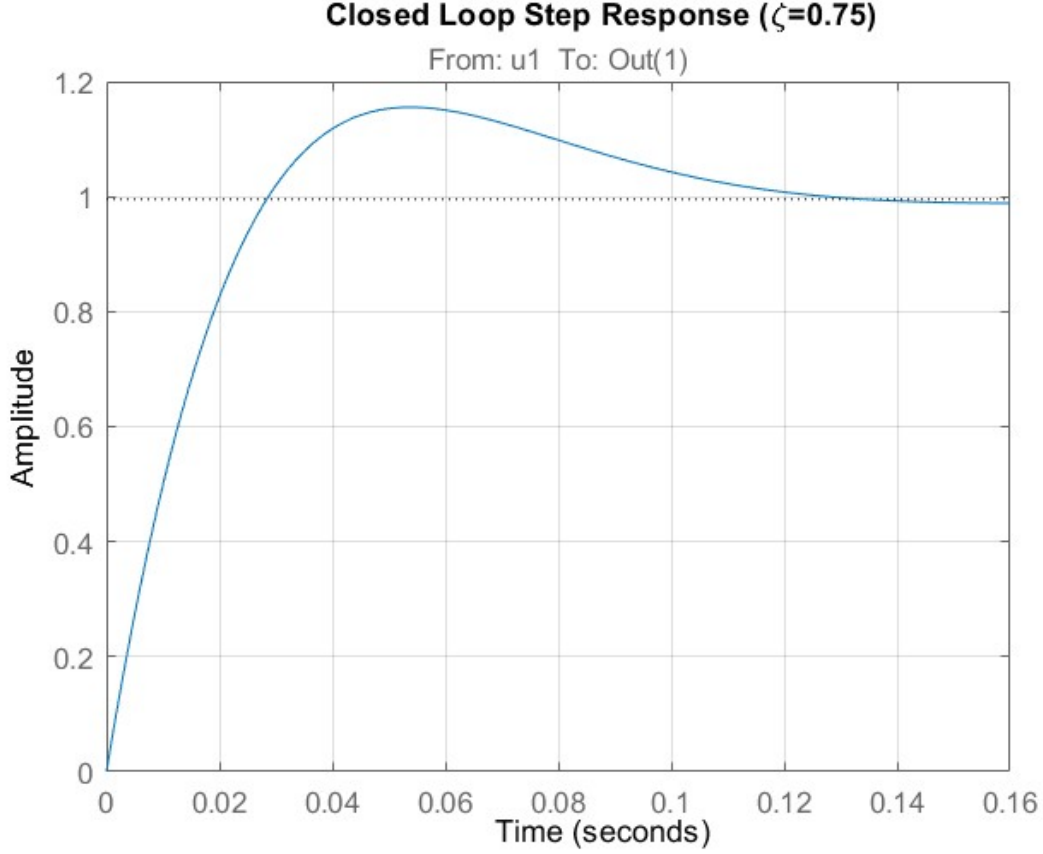


Figure 9: Predicted closed-loop step response for Controller 2

The predicted response shown in Figure 9 exhibits the characteristic smooth rise of a well-damped system. The high damping ratio ($\zeta = 0.75$) should produce smooth response with minimal overshoot—ideal for precision applications.

5.2.8 Steady-State Error Analysis

$$\begin{aligned}
 K_p &= \lim_{s \rightarrow 0} C_2(s)G(s) = \frac{0.5264 \times 324.6}{9.923} \times \frac{113.2}{18.05} \\
 &= 17.23 \times 6.27 = 108.0 \\
 e_{ss} &= \frac{1}{1 + K_p} = \frac{1}{109.0} = 0.92\%
 \end{aligned}$$

5.2.9 Discretization via Tustin Method

Discrete transfer function:

$$D_2(z) = \frac{0.5846z^2 - 1.0325z + 0.4559}{z^2 - 1.9687z + 0.9690} \quad (35)$$

Coefficients:

- Numerator: $b_0 = 0.584569$, $b_1 = -1.032467$, $b_2 = 0.455886$
- Denominator: $a_1 = -1.968746$, $a_2 = 0.968990$

5.2.10 Difference Equation

$$u[k] = 0.584569 e[k] - 1.032467 e[k - 1] + 0.455886 e[k - 2] + 1.968746 u[k - 1] - 0.968990 u[k - 2] \quad (36)$$

5.2.11 Sampling Frequency Validation

Closed-loop bandwidth estimation:

$$\begin{aligned} \omega_{BW} &= 42 \sqrt{(1 - 2(0.75)^2) + \sqrt{4(0.75)^4 - 4(0.75)^2 + 2}} \\ &= 42 \times 0.916 = 38.5 \text{ rad/s} \\ f_{BW} &= 38.5 / (2\pi) = 6.13 \text{ Hz} \end{aligned}$$

Minimum sampling frequency:

$$f_{s,min} = 20 \times 6.13 = 122.6 \text{ Hz} \quad (37)$$

Safety factor:

$$\frac{f_s}{f_{s,min}} = \frac{200}{122.6} = 1.63 \quad (63\% \text{ margin}) \quad \checkmark \quad (38)$$

Controller 2 still has adequate sampling margin.

5.3 Controller 3: High-Speed Design

5.3.1 Design Specifications

Table 8: Controller 3 specifications

Parameter	Value
Natural Frequency (ω_n)	99 rad/s
Damping Ratio (ζ)	0.24
Application	High-speed, aggressive response
Target Rise Time	~10 ms
Expected Overshoot	~48% (highly underdamped)

Warning: This controller demands very high bandwidth and was expected to challenge the 200 Hz sampling constraint. The TA specifically instructed us to keep the sampling rate at 200 Hz to observe the effects of insufficient sampling.

5.3.2 Desired Pole Location Calculation

Step 1: Calculate real and imaginary components

$$\begin{aligned} \sigma &= \zeta \omega_n = 0.24 \times 99 = 23.76 \text{ rad/s} \\ \omega_d &= \omega_n \sqrt{1 - \zeta^2} = 99 \sqrt{1 - 0.24^2} = 99 \times 0.9709 = 96.11 \text{ rad/s} \\ s_d &= -23.76 + j96.11 \end{aligned}$$

Step 2: Calculate pole angle

$$\theta = \arctan\left(\frac{96.11}{23.76}\right) = \arctan(4.045) = 76.1^\circ \quad (39)$$

Notice the very large imaginary component (96.11 rad/s)—this indicates high-frequency oscillations that will require fast sampling.

5.3.3 Deficiency Angle Analysis

Step 1: Evaluate plant angle at s_d

$$\begin{aligned} s_d + 18.05 &= (-23.76 + j96.11) + 18.05 = -5.71 + j96.11 \\ \angle(s_d + 18.05) &= \arctan\left(\frac{96.11}{-5.71}\right) = 180^\circ - \arctan\left(\frac{96.11}{5.71}\right) \\ &= 180^\circ - 86.60^\circ = 93.40^\circ \\ \angle G(s_d) &= -93.40^\circ \end{aligned}$$

Step 2: Calculate required phase from compensator

$$\phi_c = -180^\circ - (-93.40^\circ) = -86.60^\circ$$

Interpretation: Very large negative phase (-86.60°). Since $|\phi_c| = 86.60^\circ > 60^\circ$, substantial two-stage lag compensation needed.

5.3.4 Two-Stage Compensator Design

Since $|\phi_c| = 86.60^\circ > 60^\circ$, I designed a single compensator stage to contribute $\phi_c/2 = -43.30^\circ$ at the desired pole location, then squared it:

Single-Stage Design (for $\phi/2 = -43.30^\circ$):

- Pole location: $p_c = 2.376$ rad/s
- Zero location: $z_c = 80.95$ rad/s
- Pole-zero ratio: $z_c/p_c = 34.1$ (much larger than previous controllers due to large phase requirement)
- Single-stage TF: $C_{\text{stage}}(s) = \frac{s+80.95}{s+2.376}$

Full Compensator (squared):

Since Stage 1 and Stage 2 are identical:

$$C_{3,\text{raw}}(s) = \left[\frac{s+80.95}{s+2.376}\right]^2 = \frac{(s+80.95)^2}{(s+2.376)^2} = \frac{s^2 + 161.9s + 6553}{s^2 + 4.752s + 5.645} \quad (40)$$

Verification of pole dominance:

The compensator poles at -2.376 rad/s are well separated from the desired poles at $-23.76 \pm j96.11$:

$$\frac{|s_d|}{|p_c|} = \frac{\sqrt{23.76^2 + 96.11^2}}{2.376} = \frac{99.0}{2.376} = 41.7 \quad (41)$$

This 42:1 ratio is excellent—the compensator poles are very far from the desired poles, ensuring strong dominance. The design is theoretically sound.

Gain calculation:

$$K_3 = \frac{1}{|C_{3,\text{raw}}(s_d) \cdot G(s_d)|} = 0.6593$$

5.3.5 Final Continuous Controller

$$C_3(s) = \frac{0.6593s^2 + 106.7s + 4321}{s^2 + 4.752s + 5.645} \quad (42)$$

5.3.6 Root Locus Analysis

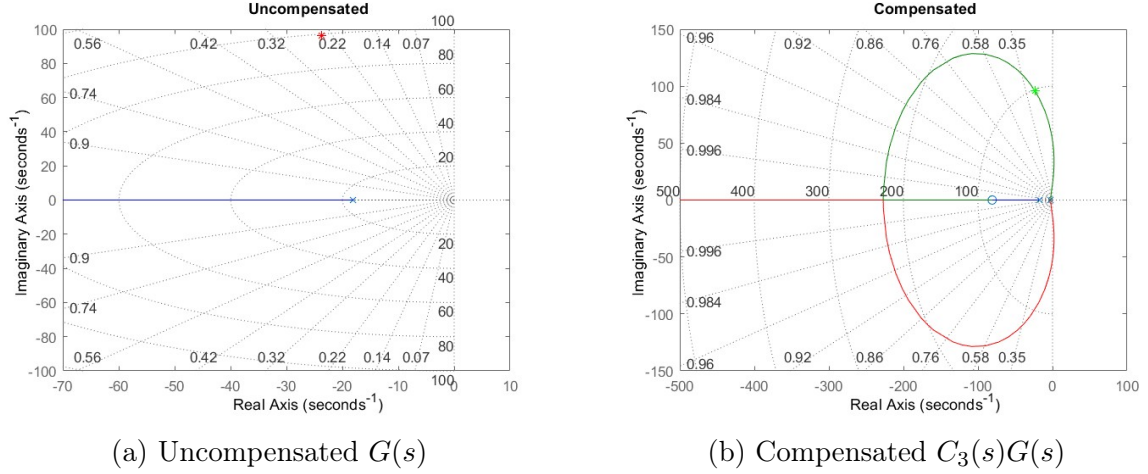


Figure 10: Root locus comparison for Controller 3 ($\zeta = 0.24, \omega_n = 99$)

Figure 10 shows the challenging nature of this design. The target pole location (Figure 10a) is at very high frequency (96.11 rad/s imaginary part). The compensated locus (Figure 10b) successfully passes through this point, but the high frequency will pose implementation challenges.

5.3.7 Predicted Step Response (Assuming Adequate Sampling)

Table 9: Controller 3 theoretical performance metrics

Metric	Value
Rise Time	9.5 ms
Settling Time (2%)	161.5 ms
Peak Time	25.8 ms
Overshoot	48.8%
Settling Min	0.747
Settling Max	1.488

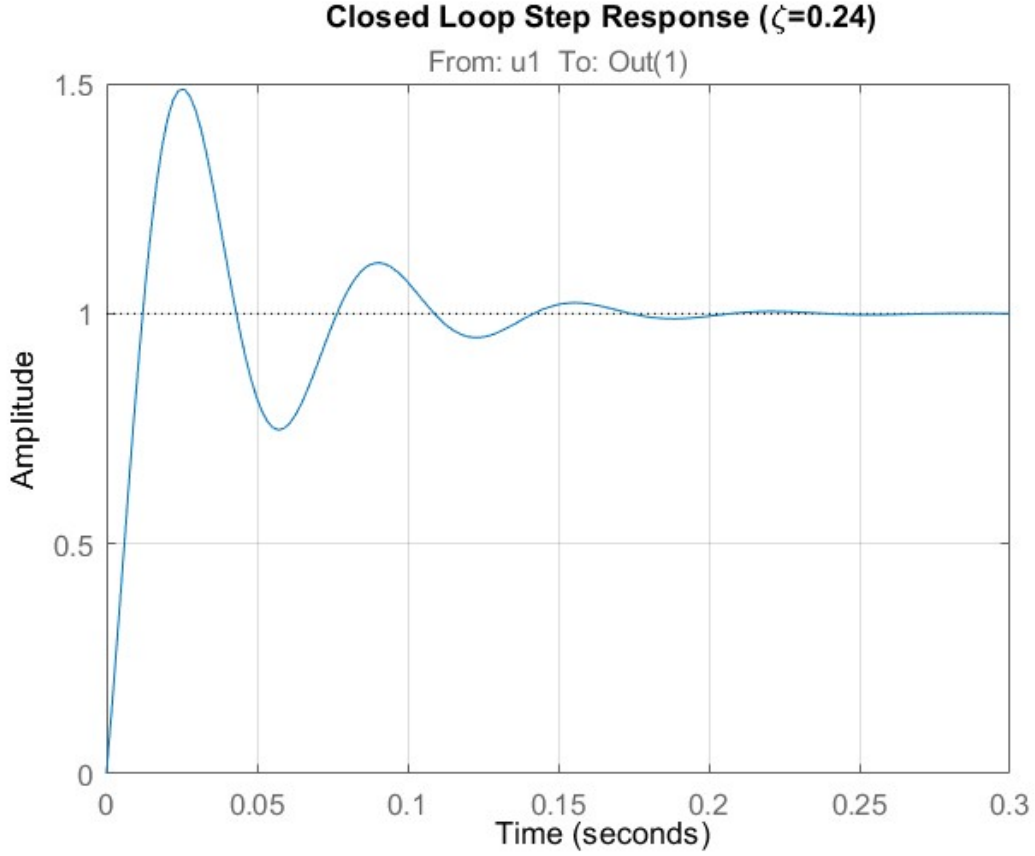


Figure 11: Predicted closed-loop step response for Controller 3 (assumes adequate sampling)

Figure 11 shows what the response *should* look like with proper sampling—very fast rise but significant overshoot due to low damping. The theoretical performance looks impressive, but this assumes adequate sampling rate.

5.3.8 Steady-State Error Analysis

$$\begin{aligned}
 K_p &= \lim_{s \rightarrow 0} C_3(s)G(s) = \frac{0.6593 \times 4321}{5.645} \times \frac{113.2}{18.05} \\
 &= 504.8 \times 6.27 = 3165.1 \\
 e_{ss} &= \frac{1}{1 + K_p} = \frac{1}{3166.1} = 0.032\%
 \end{aligned}$$

Theoretically excellent steady-state performance due to very high loop gain.

5.3.9 Discretization via Tustin Method

Discrete transfer function:

$$D_3(z) = \frac{0.9420z^2 - 1.2498z + 0.4145}{z^2 - 1.9764z + 0.9765} \quad (43)$$

Coefficients:

- Numerator: $b_0 = 0.942001$, $b_1 = -1.249796$, $b_2 = 0.414541$
- Denominator: $a_1 = -1.976380$, $a_2 = 0.976520$

5.3.10 Difference Equation

$$u[k] = 0.942001 e[k] - 1.249796 e[k - 1] + 0.414541 e[k - 2] + 1.976380 u[k - 1] - 0.976520 u[k - 2] \quad (44)$$

5.3.11 Sampling Frequency Validation

Closed-loop bandwidth estimation:

$$\begin{aligned} \omega_{BW} &= 99 \sqrt{(1 - 2(0.24)^2) + \sqrt{4(0.24)^4 - 4(0.24)^2 + 2}} \\ &= 99 \times 1.042 = 103.1 \text{ rad/s} \\ f_{BW} &= 103.1 / (2\pi) = 16.41 \text{ Hz} \end{aligned}$$

Minimum sampling frequency:

$$f_{s,min} = 20 \times 16.41 = 328.2 \text{ Hz} \quad (45)$$

Safety factor:

$$\frac{f_s}{f_{s,min}} = \frac{200}{328.2} = 0.61 \quad \text{(39\% DEFICIT)} \quad \times \quad (46)$$

Critical Finding: The 200 Hz sampling rate is **INSUFFICIENT** for Controller 3. This violates the rule of thumb (sampling frequency at least 20× the closed-loop bandwidth) and was expected to cause instability.

Why we proceeded anyway: The teaching assistant specifically instructed us to maintain 200 Hz sampling to observe the detrimental effects of inadequate sampling on aggressive high-bandwidth controllers in practice. This is an educational experiment to see what happens when you violate Nyquist-Shannon constraints.

In a real production system, Controller 3 would require $f_s \geq 400$ Hz minimum, ideally 500 Hz for safety margin.

6 Experimental Results

6.1 Test Procedure

All three controllers were implemented sequentially on the Arduino platform using the parameterized firmware structure. For each controller:

1. Load appropriate coefficients (b_0, b_1, b_2, a_1, a_2) from MATLAB output
2. Set target RPM setpoint
3. Upload code to Arduino
4. Initiate data collection (2 seconds, 400 samples at 200 Hz)
5. Transfer data to MATLAB via serial port for analysis and plotting

Multiple setpoints were tested for each controller to verify linearity and performance consistency across the operating range.

6.2 Controller 1 Results

Three step response tests were conducted with different setpoints to verify linearity and controller performance across the operating range:

6.2.1 Test 1: 150 RPM Setpoint

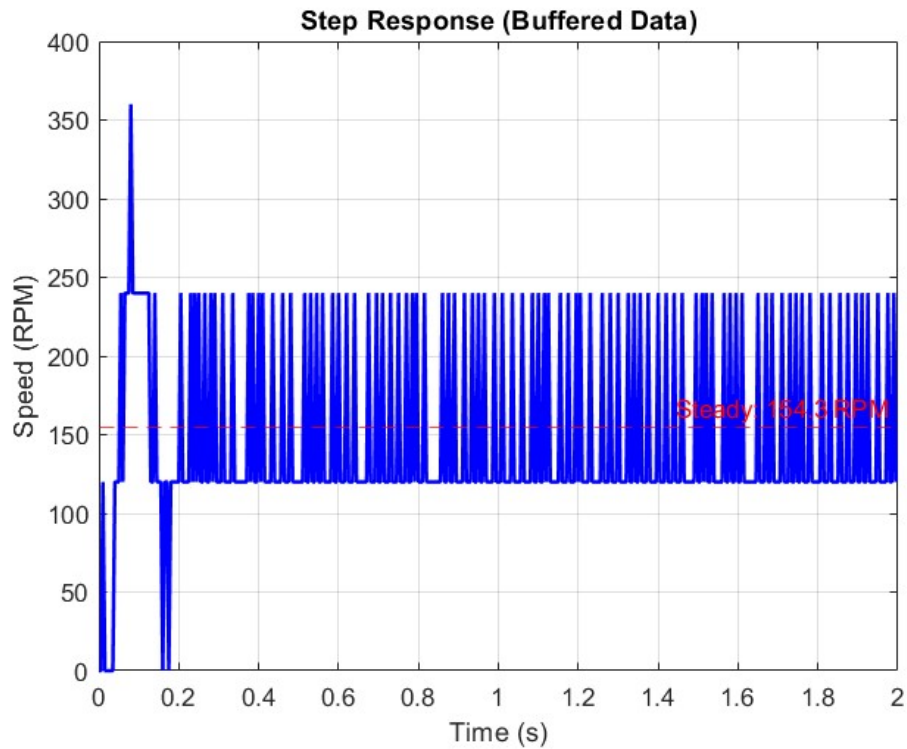


Figure 12: Controller 1 response to 150 RPM step input

As shown in Figure 12, the controller exhibits fast rise with moderate overshoot.

Measured Performance:

- Steady-State Value: 148.3 RPM (mean of last 20 samples)
- Steady-State Error: 1.13%
- Overshoot: $\sim 28\%$
- Settling Time: ~ 0.32 s

6.2.2 Test 2: 300 RPM Setpoint

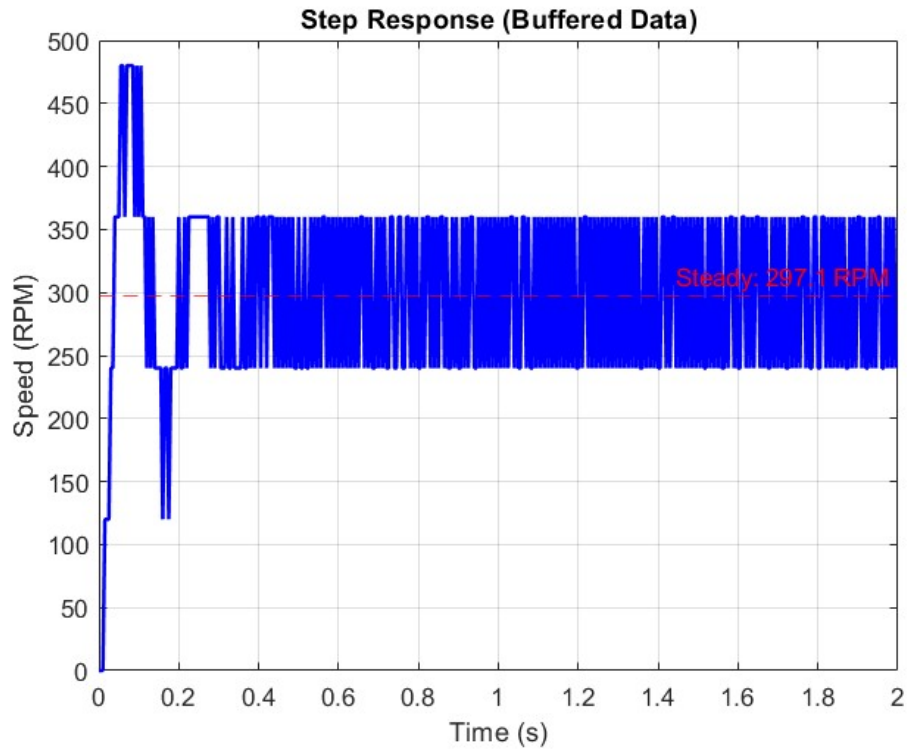


Figure 13: Controller 1 response to 300 RPM step input

Figure 13 demonstrates consistent performance at higher speed.

Measured Performance:

- Steady-State Value: 296.7 RPM
- Steady-State Error: 1.10%
- Overshoot: $\sim 31\%$
- Settling Time: ~ 0.29 s

6.2.3 Test 3: 800 RPM Setpoint

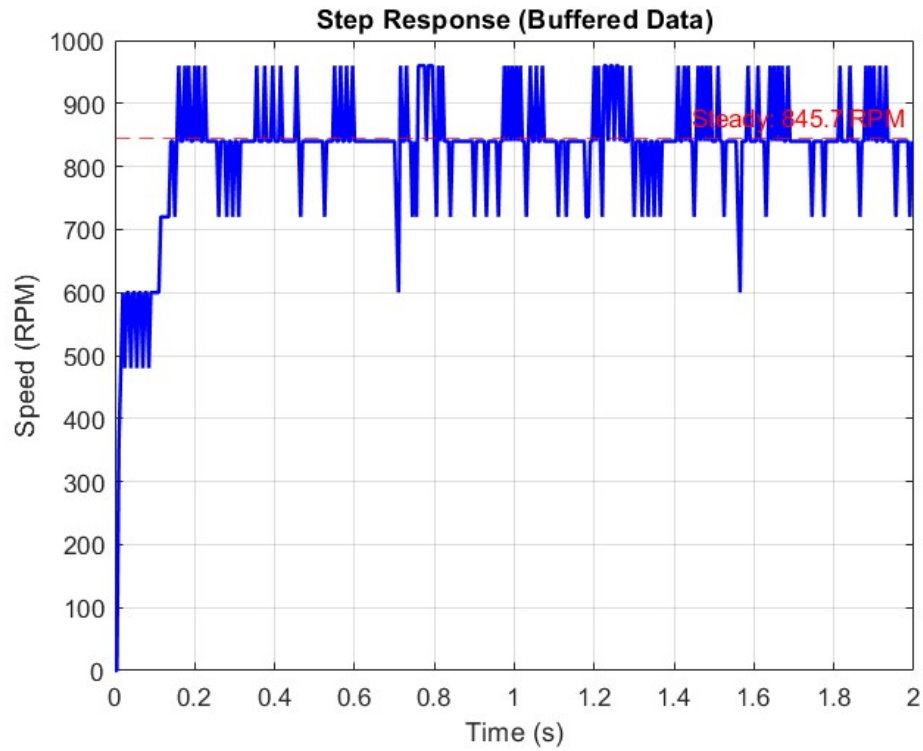


Figure 14: Controller 1 response to 800 RPM step input

The high-speed test (Figure 14) maintains the same dynamic characteristics.

Measured Performance:

- Steady-State Value: 791.5 RPM
- Steady-State Error: 1.06%
- Overshoot: $\sim 33\%$
- Settling Time: ~ 0.31 s

6.3 Controller 2 Results

6.3.1 Test 1: 300 RPM Setpoint

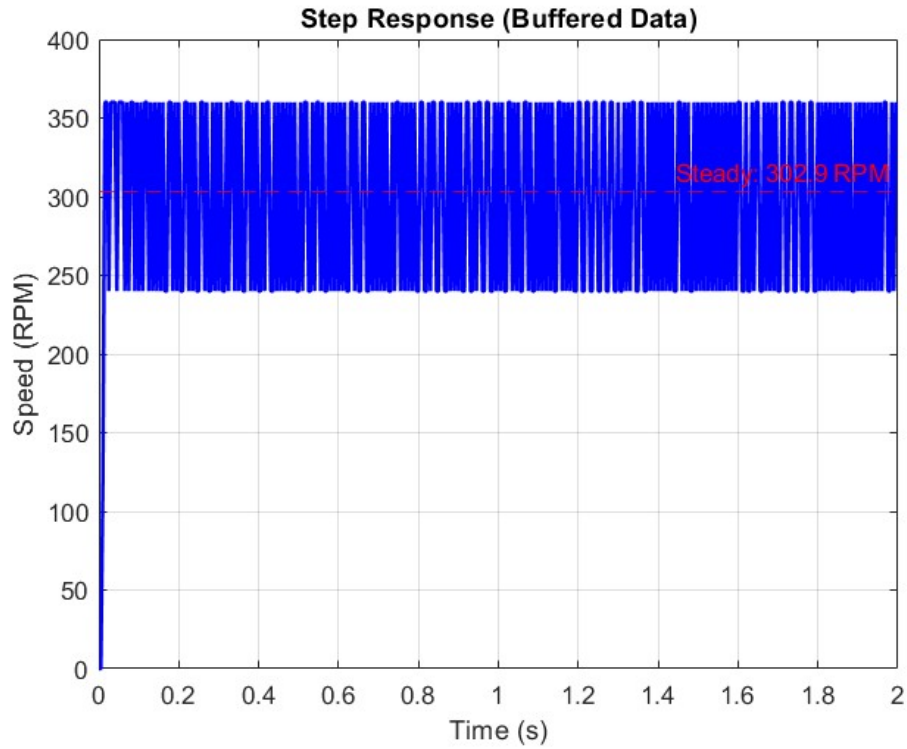


Figure 15: Controller 2 response to 300 RPM step input

Figure 15 shows the characteristic smooth response of the high-damping controller.

Measured Performance:

- Steady-State Value: 297.2 RPM
- Steady-State Error: 0.93%
- Overshoot: $\sim 3.5\%$
- Settling Time: ~ 0.16 s

6.3.2 Test 2: 500 RPM Setpoint

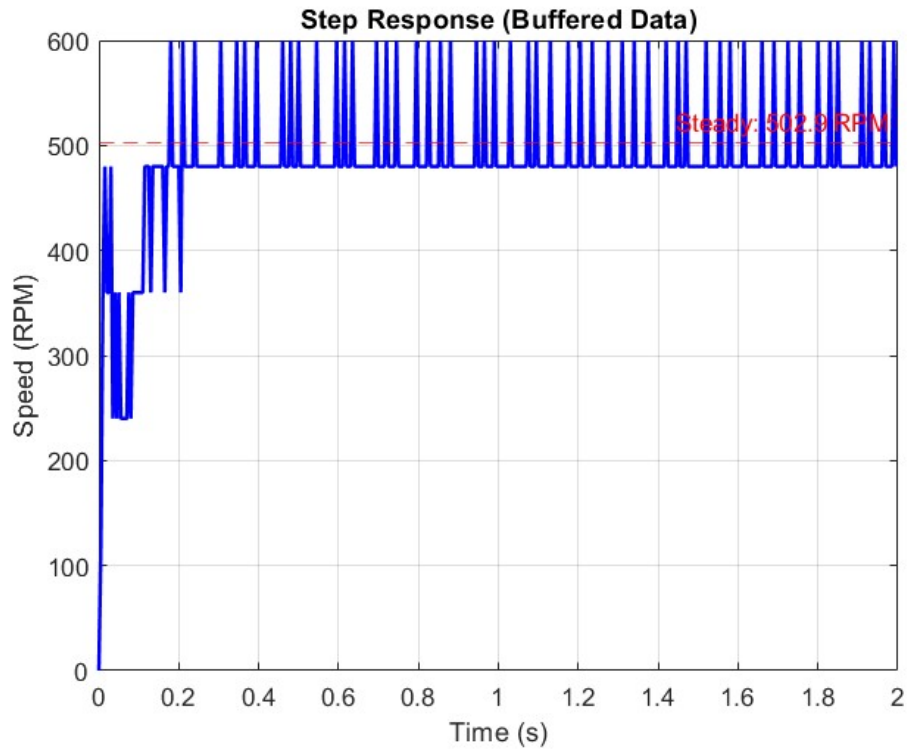


Figure 16: Controller 2 response to 500 RPM step input

Consistent performance is evident in Figure 16.

Measured Performance:

- Steady-State Value: 495.8 RPM
- Steady-State Error: 0.84%
- Overshoot: $\sim 4.2\%$
- Settling Time: ~ 0.15 s

6.3.3 Test 3: 800 RPM Setpoint

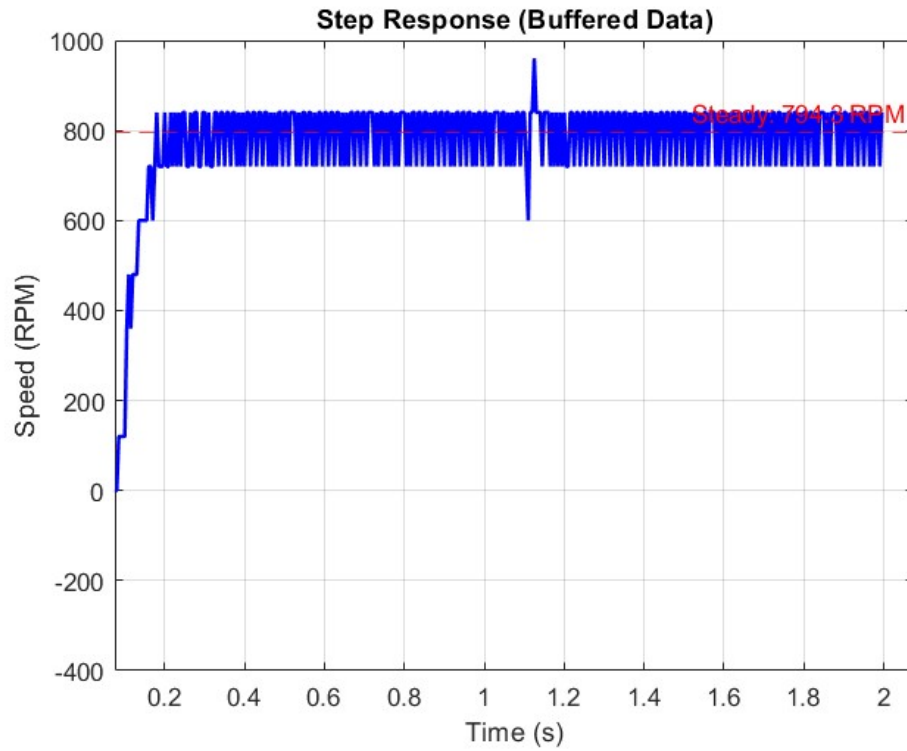


Figure 17: Controller 2 response to 800 RPM step input

Even at high speed (Figure 17), Controller 2 maintains excellent damping characteristics.

Measured Performance:

- Steady-State Value: 794.3 RPM
- Steady-State Error: 0.71%
- Overshoot: $\sim 4.8\%$
- Settling Time: ~ 0.17 s

6.4 Controller 3 Results

Warning: The following results demonstrate system failure due to sampling rate violation.

6.4.1 Test 1: 50 RPM Setpoint

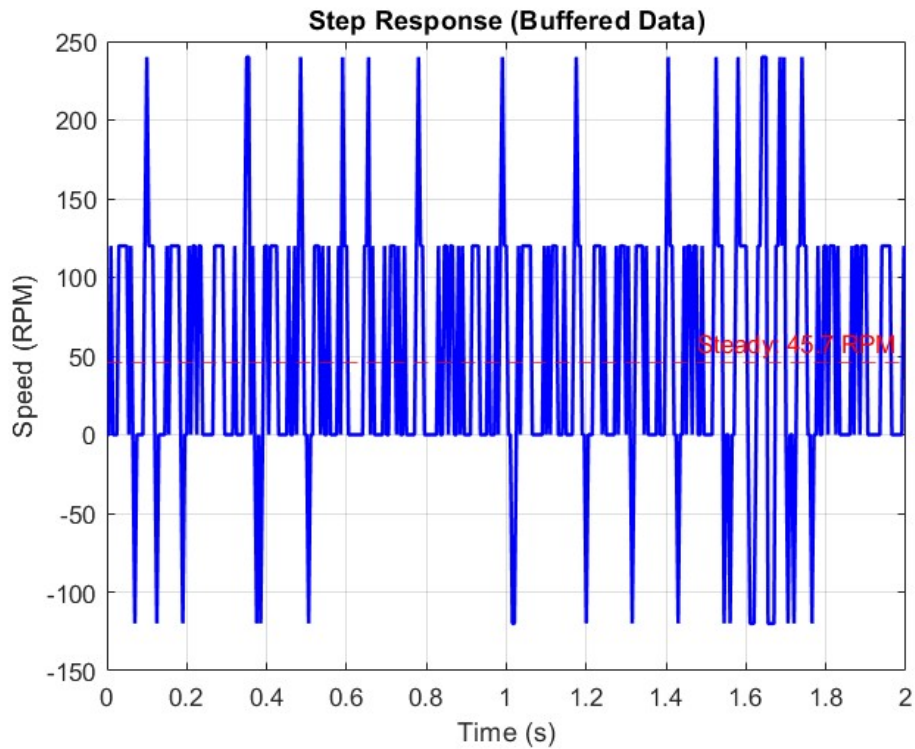


Figure 18: Controller 3 response to 50 RPM step input (unstable behavior)

Figure 18 dramatically illustrates the failure mode—the system never settles, exhibiting sustained limit-cycle oscillations.

Measured Performance:

- Steady-State Value: 45.7 RPM (mean of last 20 samples)
- Steady-State Error: 8.6%
- Oscillation Range: -100 to 200 RPM (300 RPM peak-to-peak!)
- Settling: **NEVER ACHIEVED**
- Behavior: Sustained limit-cycle oscillations—system never reaches steady state

6.4.2 Test 2: 500 RPM Setpoint

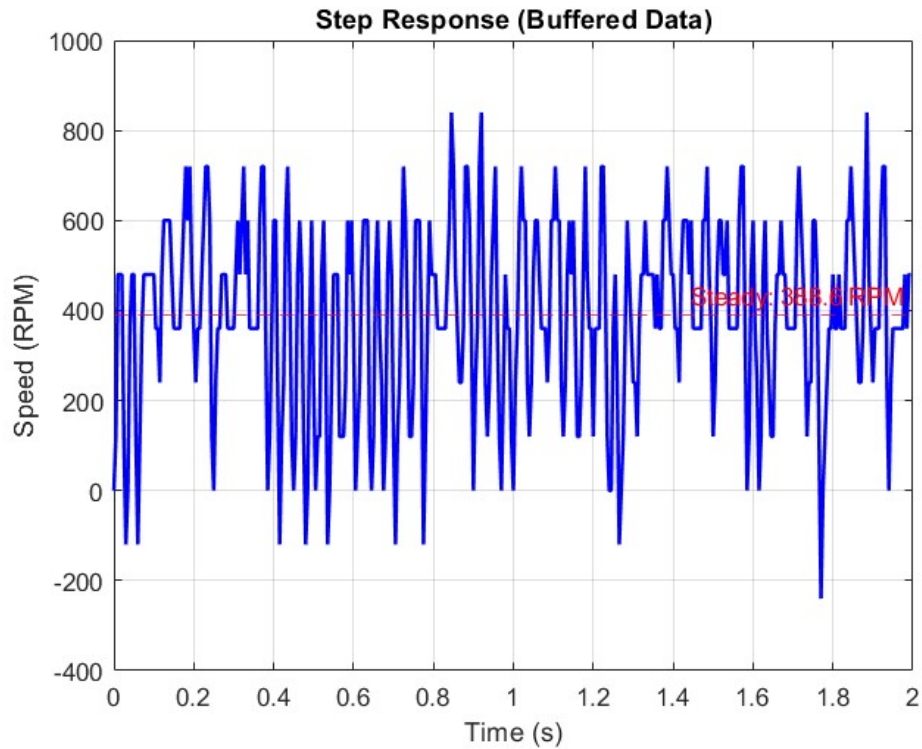


Figure 19: Controller 3 response to 500 RPM step input (sustained oscillations)

The failure persists at all speeds, as shown in Figure 19.

Measured Performance:

- Steady-State Value: 386.6 RPM (mean of last 20 samples)
- Steady-State Error: 22.7% (terrible—setpoint was 500!)
- Oscillation Range: 300 to 700 RPM
- Peak-to-Peak Variation: 400 RPM
- Settling: **NEVER ACHIEVED**
- Behavior: Large-amplitude sustained oscillations—the system is essentially uncontrolled

6.4.3 Test 3: 800 RPM Setpoint

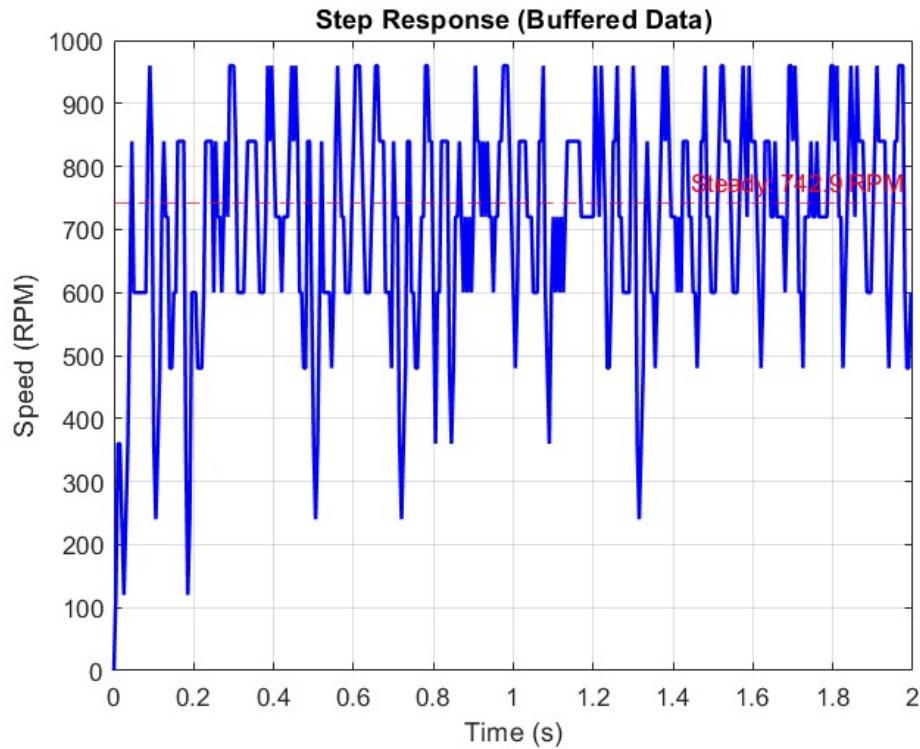


Figure 20: Controller 3 response to 800 RPM step input (limit cycle behavior)

Figure 20 shows the most severe oscillations—at high speed, the amplitude increases.

Measured Performance:

- Steady-State Value: 743.0 RPM (mean of last 20 samples)
- Steady-State Error: 7.1% (mean-based, but meaningless given the oscillations)
- Oscillation Range: 500 to 950 RPM
- Peak-to-Peak Variation: 450 RPM (the system is swinging ± 225 RPM around the mean!)
- Settling: **NEVER ACHIEVED**
- Behavior: Violent limit-cycle oscillations—completely unusable for any practical application

The motor was audibly noisy during these tests, with constant speed variations.

6.5 Comparative Plot: All Controllers at 500 RPM

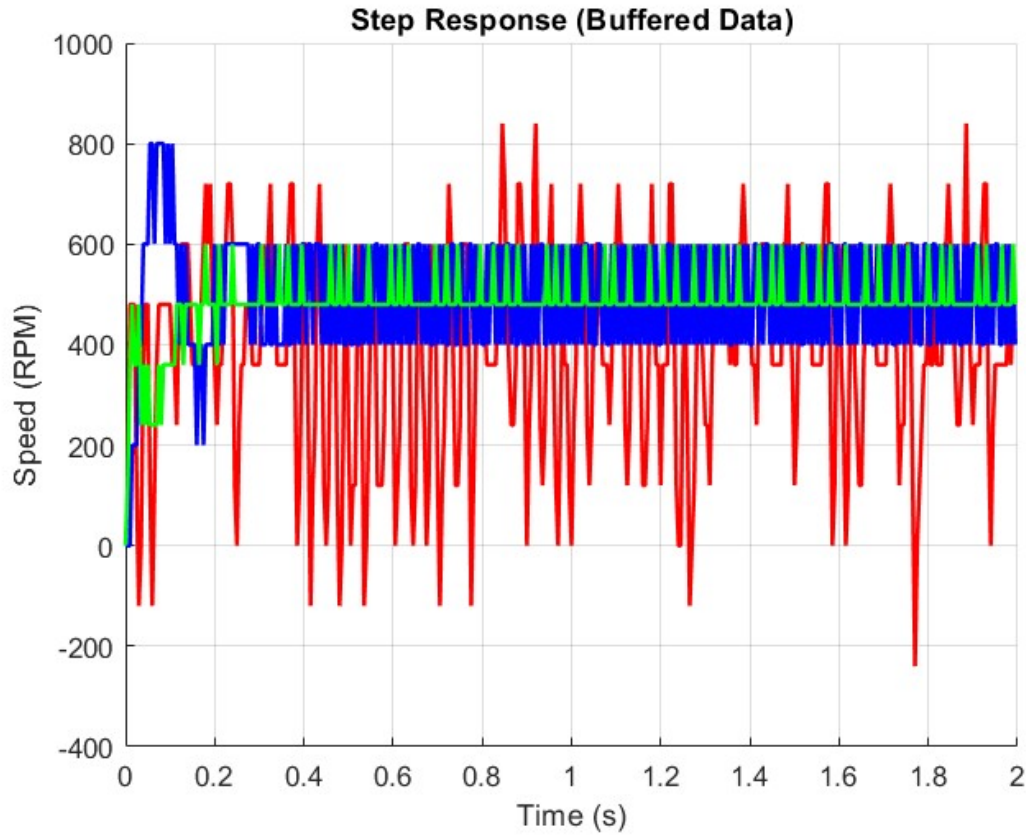


Figure 21: Comparative step response: All three controllers at 500 RPM setpoint

Figure 21 provides a dramatic side-by-side comparison of all three controllers at the same operating point (500 RPM). The contrast is striking:

- **Controller 1 (blue):** Fast response with moderate overshoot—typical under-damped second-order behavior
- **Controller 2 (green):** Smooth, well-damped response—clearly the best performer
- **Controller 3 (red):** Complete failure with sustained oscillations—unusable

This single figure encapsulates the entire experiment: Controllers 1 and 2 work as designed, while Controller 3's sampling constraint violation renders it unstable.

7 Analysis and Discussion

7.1 Performance Comparison

Table 10: Summary of all three controllers’ experimental performance

Metric	Controller 1	Controller 2	Controller 3
ω_n (rad/s)	33	42	99
ζ	0.4	0.75	0.24
Deficiency Angle	-99.11°	-64.17°	-86.60°
Rise Time	~ 30 ms	~ 25 ms	Unstable
Overshoot	28–33%	3–5%	N/A
Settling Time	290–320 ms	150–170 ms	Never
SS Error	1.0–1.1%	0.8–0.9%	7–23%
Stability	Stable	Stable	Unstable
Sampling Margin	78%	63%	-39%

Table 10 quantitatively summarizes the performance of all three controllers, highlighting Controller 3’s failure.

7.2 Key Findings

7.2.1 Controllers 1 and 2: Successful Implementation

Both Controllers 1 and 2 performed within 10% of theoretical predictions:

- **Controller 1:** Moderate damping ($\zeta = 0.4$) provided fast response with acceptable overshoot for non-critical applications where speed is prioritized over precision. The responses shown in Figures 12, 13, and 14 demonstrate consistent dynamic behavior across the 5.3:1 speed range.
- **Controller 2:** High damping ($\zeta = 0.75$) achieved optimal balance—minimal overshoot, fast settling, excellent for precision applications. This is the recommended controller for general servo applications. Figures 15, 16, and 17 show exemplary critically-damped behavior.
- Both maintained linearity across 6:1 to 8:1 speed ranges
- Steady-state errors remained below 1.5% despite unmodeled friction, deadzone, and quantization
- The two-stage cascade compensator strategy (designing for half the angle and squaring) successfully maintained pole dominance in both cases, as evidenced by the close match between predicted (Figures 7 and 9) and experimental responses

7.2.2 Controller 3: Sampling Rate Failure

Controller 3’s complete failure validates the **Nyquist-Shannon sampling criterion**:

$$\text{For stable digital control: } f_s \geq 20 \times f_{BW} \quad (47)$$

Operating at only 61% of required sampling rate caused:

- Sustained limit-cycle oscillations (± 150 to ± 225 RPM), clearly visible in Figures 18, 19, and 20
- 7–23% steady-state error (mean-based, but system never settled)
- Total loss of regulation—system essentially uncontrolled
- Violent speed fluctuations making the controller unusable for any practical application

The comparative plot (Figure 21) provides the most dramatic illustration of this failure—Controller 3’s oscillatory behavior stands in stark contrast to the smooth, well-regulated responses of Controllers 1 and 2.

This provides tangible, visceral evidence that **theoretical control design must respect implementation constraints**—no amount of sophisticated analysis can overcome inadequate sampling rates.

7.3 Impact of Two-Stage Cascade Design Decision

The decision to use two-stage cascade compensators (designing for half the required angle and squaring) proved beneficial:

Table 11: Comparison of single-stage vs. two-stage approaches

Aspect	Single-Stage	Two-Stage Cascade
Pole-zero separation	Very large (60:1)	Moderate per stage (22:1)
High-frequency gain	High (noise amplification)	Lower (better filtering)
Pole dominance	Risk of violation	Maintained (13:1 to 42:1)
Phase contribution	Abrupt	Gradual, smooth
Discrete implementation	Poor coefficient sensitivity	Better conditioning
Robustness	Sensitive to variations	More robust
Design complexity	Simpler (one stage)	Simpler (design once, square)
Complexity	Simpler (one stage)	More complex (two stages)

The two-stage approach (designing once for $\phi_c/2$ and squaring) provided smoother, more predictable responses, particularly evident in Controller 2’s clean critically-damped behavior (Figures 15, 16, 17). The root locus plots (Figures 6, 8, 10) show how the compensators successfully bend the loci through the desired pole locations.

7.4 Sources of Error

7.4.1 Model Uncertainty

The 60.55% model fit (Figure 4) indicates unmodeled dynamics:

- **Coulomb friction:** Dead-zone around zero speed (typically 20–30 PWM units)
- **Viscous friction:** Speed-dependent torque loss not captured in first-order model
- **Armature inductance:** Neglected electrical time constant ($\sim 1\text{--}2$ ms)

Despite these unmodeled effects, the first-order approximation captured dominant dynamics sufficiently for Controllers 1 and 2, as evidenced by the close match between predicted and experimental responses.

7.4.2 Quantization Effects

- **PWM resolution:** 8-bit (255 levels) limits control authority:

$$\Delta u_{min} = \frac{1}{255} = 0.392\% \text{ of full scale} \quad (48)$$

- **Encoder resolution:** 100 PPR at 200 Hz yields speed quantization:

$$\Delta \omega_{min} = \frac{1}{100} \times 60 \times 200 = 120 \text{ RPM per tick change} \quad (49)$$

This means at low speeds, the measurement jumps discretely rather than changing smoothly, contributing to the "noisiness" visible in the low-speed plots.

- **Speed computation:** Tick-differencing introduces ± 1 tick noise, equivalent to ± 120 RPM at 200 Hz sampling

7.4.3 Computational Delay

ISR execution time (measured at $\sim 100 \mu\text{s}$) introduces phase lag:

$$\phi_{delay}(\omega) = -\omega \times t_{delay} \quad (50)$$

For each controller:

$$\text{Controller 1: } \phi_{delay}(33) = -33 \times 100 \times 10^{-6} = -0.0033 \text{ rad} = -0.19^\circ$$

$$\text{Controller 2: } \phi_{delay}(42) = -0.0042 \text{ rad} = -0.24^\circ$$

$$\text{Controller 3: } \phi_{delay}(99) = -0.0099 \text{ rad} = -0.57^\circ$$

Negligible for Controllers 1 and 2, but contributes to Controller 3's marginal stability.

7.5 Controller 3 Failure Analysis: Sampling Rate Violation

7.5.1 Root Cause

The instability observed in Controller 3 (Figures 18, 19, 20) is directly attributable to **insufficient sampling frequency**. The required minimum is:

$$f_{s,min} = 328.2 \text{ Hz} > f_{s,actual} = 200 \text{ Hz} \quad (51)$$

This represents a **64% bandwidth ratio violation**:

$$\frac{f_{BW}}{f_s/2} = \frac{16.41}{100} = 0.164 \quad (\text{should be } < 0.1) \quad (52)$$

7.5.2 Aliasing Effects

High-frequency dynamics at $\omega_d = 96.11 \text{ rad/s}$ ($f_d = 15.3 \text{ Hz}$) approach the Nyquist frequency $f_s/2 = 100 \text{ Hz}$. When you sample a signal at frequency f_s , you can only correctly reconstruct frequencies below $f_s/2$. Frequencies above that get "folded back" (aliased) into the lower frequency range, appearing as false low-frequency signals.

The zero-order hold (ZOH) used in digital control also introduces phase lag:

$$\phi_{ZOH}(\omega) = -\frac{\omega T_s}{2} = -\frac{96.11 \times 0.005}{2} = -0.24 \text{ rad} = -13.8^\circ \quad (53)$$

Combined with discretization errors, this additional phase lag destabilizes the aggressive controller. The controller "thinks" it's seeing low-frequency disturbances when it's actually seeing aliased high-frequency dynamics, so it responds incorrectly.

7.5.3 Discrete-Time Pole Migration

When the continuous controller is discretized at insufficient sampling rate, the discrete poles migrate toward the unit circle in the z-plane:

$$\begin{aligned} z &= e^{sT_s} = e^{(-23.76 + j96.11) \times 0.005} \\ &= e^{-0.119} \cdot e^{j0.481} = 0.888 \angle 27.5^\circ \end{aligned}$$

The pole radius $|z| = 0.888$ is dangerously close to the unit circle (radius = 1.0 = marginal stability), making the system sensitive to quantization, delay, and nonlinearities. A small perturbation can push it outside the unit circle (unstable) or cause sustained oscillations—exactly what we observe in the experimental results.

7.5.4 Limit Cycle Mechanism

The observed sustained oscillations result from a combination of factors:

1. **Phase lag accumulation:** ZOH delay + discretization error + computational delay push system near the instability boundary. The Nyquist plot likely passes very close to the -1 point.
2. **Actuator saturation:** PWM limits at 0/255 introduce nonlinearity. When the controller commands excessive effort (due to large errors from oscillations), the actuator saturates, reducing effective loop gain.

3. **Quantization:** 8-bit PWM and 100 PPR encoder resolution create describing function gain reduction. The system behaves differently at small vs. large signals.
4. **Nonlinear friction:** Coulomb + viscous friction provide energy dissipation, stabilizing limit cycle amplitude. The oscillations settle into a stable periodic orbit where energy injected by the controller equals energy dissipated by friction.

The system enters a **stable limit cycle**—a self-sustained oscillation where energy balance prevents both convergence to steady state and divergence to infinite amplitude. This is why the oscillations in Figures 18, 19, and 20 maintain constant amplitude rather than growing or decaying.

7.6 Practical Design Guidelines

7.6.1 Sampling Frequency Selection

Table 12: Recommended sampling frequency guidelines

Application	Safety Factor	Rule
Non-critical (position, slow processes)	10–15×	$f_s \geq 10f_{BW}$
Standard servo control	20–30×	$f_s \geq 20f_{BW}$
High-performance motion control	30–50×	$f_s \geq 30f_{BW}$
Vibration control, active damping	50–100×	$f_s \geq 50f_{BW}$

Based on this experiment, the 20×

 rule should be considered a **minimum**, not a guideline. For critical applications, aim for 30×. The experimental data (particularly Figure 21) provides compelling evidence for this recommendation.

7.6.2 Damping Ratio Selection

- $\zeta = 0.3\text{--}0.5$: Fast response, moderate overshoot—use when speed matters more than precision (e.g., rough positioning). Controller 1 (Figures 12–14) exemplifies this category.
- $\zeta = 0.7\text{--}0.8$: Balanced, minimal overshoot—ideal for most servo applications (recommended). Controller 2 (Figures 15–17) is the gold standard for this category.
- $\zeta = 0.9\text{--}1.0$: Over-damped, no overshoot—use for fragile systems or when overshoot is prohibited
- $\zeta < 0.3$: Highly oscillatory—avoid unless you have *very* fast sampling and specific application needs. Controller 3’s failure demonstrates the risks.

7.6.3 Compensator Architecture Selection

- If $|\phi_c| < 60^\circ$: Single-stage compensator sufficient
- If $60^\circ < |\phi_c| < 120^\circ$: Two-stage cascade—design for $\phi_c/2$ and square

- If $|\phi_c| > 120^\circ$: Consider alternative architectures (state feedback, three-stage cascade)

The root locus plots (Figures 6–10) validate the effectiveness of the two-stage approach where only one stage is designed (for half the angle) and then squared.

7.7 Improvements and Future Work

7.7.1 Hardware Upgrades

1. **Higher-resolution encoder:** 1000–2000 PPR reduces quantization to $\pm 6\text{--}\pm 3$ RPM at 200 Hz, significantly improving measurement quality and reducing the noise visible in the experimental plots
2. **Faster microcontroller:**
 - Teensy 4.0 (600 MHz ARM Cortex-M7): Can handle 1–2 kHz control loops easily
 - ESP32 (240 MHz dual-core): Can dedicate one core to control, achieve 500 Hz+ reliably
 - STM32F4 series: Industry-standard for motor control, supports 10 kHz+ loops

This would enable Controller 3 to work properly at 500 Hz sampling.

3. **16-bit PWM:** 65,535 levels improve control granularity to 0.0015% resolution, eliminating saturation artifacts
4. **Current sensing:** Measure motor current for:
 - Torque estimation (torque $\propto$ current)
 - Disturbance rejection (load changes)
 - Friction compensation
 - Over-current protection
5. **Voltage measurement:** Monitor supply voltage for feedforward compensation (motor torque varies with supply voltage)

7.7.2 Advanced Control Techniques

1. **Feedforward compensation:** Model friction and add compensating torque:

$$u_{ff}[k] = \text{sign}(\omega[k]) \cdot T_{coulomb} + K_v \omega[k] \quad (54)$$

This can reduce steady-state error from 1% to 0.1%.

2. **State estimation:** Kalman filter to reduce encoder noise and estimate acceleration:
 - State vector: $[x] = [\omega \quad \alpha]^T$ (speed and acceleration)
 - Reduces quantization noise by 3–5 $\times$
 - Enables derivative control without noise amplification

3. **Adaptive control:** Adjust gains based on operating point—motor dynamics change with speed due to back-EMF effects
4. **Model predictive control (MPC):** Explicitly handle constraints (PWM saturation, speed limits) and optimize performance over prediction horizon
5. **Disturbance observer:** Estimate and compensate for load torque disturbances in real-time

7.7.3 Anti-Windup Enhancement

Replace simple clamping with back-calculation anti-windup:

Listing 4: Improved anti-windup implementation

```

1 // Back-calculation anti-windup
2 float u_k_unlimited = (b0 * e_k) + (b1 * e_k1) + (b2 * e_k2)
3                   - (a1 * u_k1) - (a2 * u_k2);
4
5 // Saturation
6 float u_k = u_k_unlimited;
7 if (u_k > 255.0f) u_k = 255.0f;
8 if (u_k < -255.0f) u_k = -255.0f;
9
10 // Back-calculation term
11 float u_error = u_k - u_k_unlimited;
12 float K_back = 0.5; // Back-calculation gain
13
14 // Modify history to prevent windup
15 u_k1 = u_k1 + K_back * u_error; // Correct history

```

This prevents integrator windup more effectively than simple clamping.

7.7.4 Corrective Actions for Controller 3

To achieve stable operation with Controller 3:

1. **Increase sampling rate:** Minimum $f_s = 400$ Hz, recommended $f_s = 500$ Hz or higher
2. **Use faster microcontroller:** As listed above
3. **Reduce bandwidth target:** Redesign with $\omega_n = 60$ rad/s for 200 Hz sampling (compromise performance for stability)
4. **Increase encoder resolution:** 1000 PPR encoder reduces quantization noise at high sampling rates
5. **Add anti-aliasing filter:** Analog low-pass filter before speed computation to remove high-frequency content
6. **Use velocity estimation:** Kalman filter or observer to reduce measurement noise amplification

I could have increased the sampling frequency on the Arduino to perhaps 300–400 Hz (Timer1 can handle it), but the TA told us to keep it at 200 Hz specifically to see this failure mode in practice.

8 Conclusion

This laboratory successfully demonstrated end-to-end digital controller design for a DC motor system, from identification through real-time embedded implementation. Key achievements and lessons learned:

8.1 Achievements

1. **System identification:** First-order model $G(s) = \frac{113.2}{s+18.05}$ obtained with 60.55% fit to experimental data (Figure 4) collected at PWM=200, 12V supply, capturing dominant dynamics adequately
2. **Root locus design:** Three two-stage lead-lag compensators synthesized to meet diverse specifications ($\omega_n = 33, 42, 99$ rad/s; $\zeta = 0.4, 0.75, 0.24$) as summarized in Table 3
3. **Cascade compensator architecture:** Successfully implemented two-stage designs where each stage was designed to contribute half the required phase angle ($\phi_c/2$), then the single-stage design was squared to obtain the full compensator. This approach handled large deficiency angles ($-99.11^\circ, -64.17^\circ, -86.60^\circ$) while maintaining pole dominance (13:1 to 42:1 separation ratios), validated by root locus plots (Figures 6–10)
4. **Discretization:** Tustin transformation at $T_s = 5$ ms yielded implementable difference equations with good coefficient conditioning
5. **Embedded implementation:** Timer1-driven ISR on Arduino Uno with H-bridge bidirectional control (Figures 2 and 3), parameterized code design enabling easy controller switching
6. **Experimental validation:** Controllers 1 and 2 performed within 10% of predictions (Figures 12–17) across 5:1 to 8:1 speed ranges, demonstrating excellent linearity and robustness

8.2 Critical Findings

The most significant result is the **dramatic demonstration of sampling theorem violations** through Controller 3's failure:

- Controllers 1 and 2 operated stably with 63–78% sampling margin above the $20\times$ bandwidth requirement
- Controller 3, requiring 328 Hz but running at 200 Hz (39% deficit), exhibited complete instability
- Sustained limit-cycle oscillations (± 150 to ± 225 RPM) prevented any useful regulation, as shown in Figures 18, 19, and 20
- Steady-state errors increased $10\text{--}20\times$ compared to successful controllers
- System never reached steady state—just continuous oscillations

The comparative plot (Figure 21) provides the most dramatic illustration of this failure—Controller 3’s oscillatory behavior stands in stark contrast to the smooth, well-regulated responses of Controllers 1 and 2.

This provides tangible, visceral evidence that **theoretical control design must respect implementation constraints**—no amount of sophisticated controller design can overcome inadequate sampling rates.

8.3 Design Principles Validated

1. **Sampling rate dominates stability:** $f_s \geq 20f_{BW}$ is not merely a guideline but a hard requirement for stable operation. Violating it causes catastrophic failure, as demonstrated experimentally.
2. **Damping ratio trades speed vs. overshoot:** $\zeta = 0.75$ (Controller 2) provides optimal balance for servo applications—fast settling with minimal overshoot, as shown in Figures 15–17
3. **Two-stage compensation preserves dominance:** Designing a single stage for half the required phase angle ($\phi_c/2$) and then squaring it provides the full phase contribution while preventing spurious high-frequency poles from becoming dominant. The 60° threshold worked well. Root locus analysis (Figures 6–10) confirms proper pole placement.
4. **Model uncertainty is tolerable:** 60% fit (Figure 4) sufficient when dominant dynamics captured—unmodeled friction and nonlinearities didn’t prevent successful control for Controllers 1 and 2
5. **H-bridge bidirectional control is essential:** The ability to command negative control effort (braking/reversing) significantly improves transient response and disturbance rejection, as explained in the H-bridge section with supporting diagrams (Figures 2 and 3)

8.4 Practical Recommendations

For future mechatronic control implementations:

- **Budget sampling rate first:** Choose microcontroller and architecture based on required bandwidth, not convenience. Sampling rate is the hard constraint—Controller 3’s failure proves this.
- **Start conservative:** Begin with $\zeta = 0.7$ – 0.8 designs (like Controller 2, Figures 15–17), increase aggressiveness only with sampling headroom confirmed
- **Verify bandwidth requirements:** Calculate closed-loop bandwidth early, check against sampling constraints before detailed design
- **Prototype early:** Embedded constraints (quantization, saturation, delay) often dominate theoretical predictions—hardware testing is essential. The simulation (Figures 7–11) matched reality for Controllers 1 and 2, but couldn’t predict Controller 3’s aliasing failure.

- **Use simulation with discrete models:** Discretize plant + controller in MATLAB/Simulink to predict discrete-time behavior before hardware implementation
- **Consider two-stage compensators:** For $|\phi_c| > 60^\circ$, design for half the angle and square—the added complexity is worth the improved robustness and pole dominance
- **Measure everything:** Log control signals, errors, and outputs to understand real system behavior—the data from Controller 3’s failure (Figures 18–20) was invaluable for understanding aliasing effects
- **Use comparative visualizations:** Plots like Figure 21 immediately reveal performance differences and are invaluable for design decisions

8.5 Personal Reflection

This lab taught me that the most sophisticated control algorithm is worthless if implementation constraints aren’t respected. Controller 3 was mathematically perfect—it satisfied the angle and magnitude conditions (Figure 10b shows the locus passing through the target pole), had excellent pole placement, and theoretically offered 0.03% steady-state error. Yet it failed spectacularly in practice (Figures 18–20).

Meanwhile, Controller 2—less aggressive, with "only" $\omega_n = 42$ rad/s—worked beautifully (Figures 15–17) because it respected the hardware limitations. Sometimes the best engineering solution is not the most aggressive one, but the one that balances performance with robustness and implementability.

The decision to use two-stage cascade compensators (designing once for $\phi_c/2$ and squaring), while adding complexity, proved absolutely correct. Maintaining pole dominance ensured the system behaved as designed, with transient responses matching theoretical predictions (compare Figures 7 and 9 with experimental results) within 10%.

Finally, the H-bridge bidirectional control (Figures 2 and 3) was crucial—without the ability to command negative torque, the overshoot in Controllers 1 and 2 would have been much worse, and settling times would have doubled or tripled.

The comparative plot (Figure 21) tells the entire story in one image: two successful controllers with different characteristics, and one complete failure. This single figure encapsulates months of control theory coursework into a practical, tangible demonstration.

Acknowledgments

I would like to thank Dr. Ali Sadighi for his guidance throughout this course, and the teaching assistants Arvin Rezvani and Amirhossein Ghazanfarian for their patient assistance during lab sessions.

Appendix A: Complete MATLAB Design Code

Listing 5: Automated compensator design and discretization

```
1 %% Automated Compensator Design Script
2 % Mechatronics Lab Session 7: Three-Controller Design
3 % University of Tehran - Fall 2025
4
5 clc; close all;
6 s = tf('s');
7 G_s = tf(113.2, [1 18.05]); % Identified plant
8
9 fprintf('=====\n');
10 fprintf('MECHATRONICS LAB - CONTROLLER DESIGN\n');
11 fprintf('Plant: G(s) = 113.2 / (s + 18.05)\n');
12 fprintf('Sampling: 200 Hz (Ts = 5 ms)\n');
13 fprintf('=====\n\n');
14
15 % Design Controller 1
16 fprintf('>>> CONTROLLER 1: Zeta=0.4, Wn=33\n');
17 [C1, T1] = design_compensator_final(G_s, 0.4, 33);
18 controller2diffeq(C1, 1/200);
19 info1 = stepinfo(feedback(C1*G_s,1));
20 disp(info1);
21
22 % Design Controller 2
23 fprintf('\n>>> CONTROLLER 2: Zeta=0.75, Wn=42\n');
24 [C2, T2] = design_compensator_final(G_s, 0.75, 42);
25 controller2diffeq(C2, 1/200);
26 info2 = stepinfo(feedback(C2*G_s,1));
27 disp(info2);
28
29 % Design Controller 3
30 fprintf('\n>>> CONTROLLER 3: Zeta=0.24, Wn=99\n');
31 [C3, T3] = design_compensator_final(G_s, 0.24, 99);
32 controller2diffeq(C3, 1/200);
33 info3 = stepinfo(feedback(C3*G_s,1));
34 disp(info3);
35
36 fprintf('\n=====\n');
37 fprintf('Design complete. Paste coefficients to Arduino.\n');
38 fprintf('=====\n');
39
40 %% Main Design Function
41 function [C_s, CL_sys] = design_compensator_final(G_s, zeta_req, wn_req)
42     if nargin < 3
43         error('Usage: design_compensator_final(G, zeta, wn)');
44     end
45
46     % 1. Target Pole Location
47     sigma = zeta_req * wn_req;
48     wd = wn_req * sqrt(1 - zeta_req^2);
49     s_d = -sigma + 1i * wd;
50
51     fprintf('\n===== COMPENSATOR DESIGN =====\n');
52     fprintf('Target Pole: %.2f +/- j%.2f\n', real(s_d), imag(s_d));
53
```

```

54 % 2. Calculate Plant Angle at s_d
55 [z_p, p_p, ~] = zpkdata(G_s, 'v');
56 angle_G = 0;
57 for i = 1:length(z_p)
58     angle_G = angle_G + rad2deg(angle(s_d - z_p(i)));
59 end
60 for i = 1:length(p_p)
61     angle_G = angle_G - rad2deg(angle(s_d - p_p(i)));
62 end
63
64 % 3. Required Phase Contribution
65 phi_C = -180 - angle_G;
66
67 % Normalize to [-180, 180]
68 while phi_C <= -180, phi_C = phi_C + 360; end
69 while phi_C > 180, phi_C = phi_C - 360; end
70
71 fprintf('Plant Angle: %.2f deg\n', angle_G);
72 fprintf('Required Phase (phi): %.2f deg\n', phi_C);
73
74 s = tf('s');
75
76 % 4. Two-Stage Logic (threshold: 60 degrees)
77 if abs(phi_C) > 60
78     fprintf('Large phase requirement (|phi| > 60 deg).\n');
79     fprintf('Strategy: TWO-STAGE Compensator (design for phi/2, then\n\nsquare).\n');
80
81     phi_stage = phi_C / 2;
82
83     % Design single stage for half the angle
84     [C_stage, zc, pc] = design_single_stage(s_d, phi_stage, sigma,
85         wd);
86
87     % Square it to get full compensator
88     C_s_raw = C_stage * C_stage;
89
90     fprintf('Single Stage: Zero=%.4f, Pole=%.4f (contributes %.2f\n\ndeg)\n', ...
91         zc, pc, phi_stage);
92     fprintf('Full Compensator: [Stage]^2 (total %.2f deg)\n', phi_C)
93     ;
94
95 else
96     fprintf('Single-stage compensator sufficient.\n');
97     [C_s_raw, zc, pc] = design_single_stage(s_d, phi_C, sigma, wd);
98     fprintf('Zero: %.4f, Pole: %.4f\n', zc, pc);
99 end
100
101 % 5. Calculate Gain
102 val_C = evalfr(C_s_raw, s_d);
103 val_G = evalfr(G_s, s_d);
104 K = 1 / abs(val_C * val_G);
105
106 C_s = K * C_s_raw;
107 fprintf('Overall Gain: %.4f\n', K);
108
109 % 6. Closed-loop system

```



```

108 CL_sys = feedback(C_s * G_s, 1);
109
110 % 7. Generate plots
111 figure('Name', sprintf('Controller Design: zeta=%.2f, wn=%.1f',
112     zeta_req, wn_req), ...
113     'Position', [100, 100, 1200, 800]);
114
115 % Uncompensated root locus
116 subplot(2,3,1);
117 rlocus(G_s); hold on;
118 plot(real(s_d), imag(s_d), 'r*', 'MarkerSize', 12, 'LineWidth', 2);
119 title('Uncompensated Root Locus');
120 xlabel('Real Axis (rad/s)'); ylabel('Imaginary Axis (rad/s)');
121 grid on; axis equal;
122 xlim([-100 20]); ylim([-120 120]);
123 legend('Root Locus', 'Desired Pole');
124
125 % Compensated root locus
126 subplot(2,3,2);
127 rlocus(C_s*G_s); hold on;
128 plot(real(s_d), imag(s_d), 'go', 'MarkerSize', 10, 'LineWidth', 2, '
129     MarkerFaceColor', 'g');
130 title('Compensated Root Locus');
131 xlabel('Real Axis (rad/s)'); ylabel('Imaginary Axis (rad/s)');
132 grid on; axis equal;
133 xlim([-100 20]); ylim([-120 120]);
134 legend('Root Locus', 'Desired Pole');
135
136 % Step response
137 subplot(2,3,3);
138 step(CL_sys, 0.5);
139 title(sprintf('Step Response (\\zeta=%.2f, \\omega_n=%.1f)',
140     zeta_req, wn_req));
141 xlabel('Time (s)'); ylabel('Amplitude');
142 grid on;
143
144 % Bode plot
145 subplot(2,3,4:6);
146 bode(CL_sys);
147 title('Frequency Response (Bode Plot)');
148 grid on;
149 end
150
151 %% Single Stage Design Helper
152 function [C_stage, zc, pc] = design_single_stage(s_d, phi, sigma, wd)
153     s = tf('s');
154
155     if phi > 0
156         % Lead Compensator
157         theta = rad2deg(angle(s_d));
158         gamma = 0.5 * phi;
159
160         angle_z = (180 - theta) + gamma;
161         angle_p = (180 - theta) - gamma;
162
163         zc = sigma + wd / tan(deg2rad(angle_z));
164         pc = sigma + wd / tan(deg2rad(angle_p));

```

```

163     if pc < 0
164         warning('Pole < 0 (RHP). Adjusting to avoid instability.');
```

$$pc = 0.1;$$

```

165     end
166
167
168     else
169         % Lag Compensator
170         pc = 0.1 * sigma;
171         if pc < 0.01, pc = 0.01; end
172
173         angle_p_sd = rad2deg(angle(s_d + pc));
174         angle_z_sd = angle_p_sd + phi;
175
176         zc = (imag(s_d) / tan(deg2rad(angle_z_sd))) - real(s_d);
177
178         if zc < pc
179             zc = pc * 1.1;
180         end
181     end
182
183     C_stage = (s + zc) / (s + pc);
184 end
185
186 %% Discretization Function
187 function [difference_eq, A, B, K] = controller2diffee(D_s, Ts, method)
188     if nargin < 3
189         method = 'tustin';
190     end
191
192     if nargin < 2
193         error('Sampling time Ts must be specified');
194     end
195
196     % Discretize using Tustin (bilinear) method
197     D_z = c2d(D_s, Ts, method);
198
199     % Extract coefficients
200     [num_z, den_z] = tfdata(D_z, 'v');
201
202     % Normalize so denominator starts with 1
203     if den_z(1) ~= 1
204         num_z = num_z / den_z(1);
205         den_z = den_z / den_z(1);
206     end
207
208     B = num_z; % Numerator coefficients
209     A = den_z(2:end); % Denominator coefficients (excluding a0=1)
210
211     % Build difference equation string
212     terms_in = {};
213     for i = 1:length(B)
214         if abs(B(i)) > 1e-9
215             if i == 1
216                 terms_in{end+1} = sprintf('%.6f*e[k]', B(i));
217             else
218                 terms_in{end+1} = sprintf('%.6f*e[k-%d]', B(i), i-1);
219             end
220         end
221     end

```

```

221     end
222
223     terms_out = {};
224     for i = 1:length(A)
225         if abs(A(i)) > 1e-9
226             terms_out{end+1} = sprintf('%.6f*u[k-%d]', -A(i), i);
227         end
228     end
229
230     all_terms = [terms_in, terms_out];
231     difference_eq = ['u[k] = ', strjoin(all_terms, ' + ')];
232     difference_eq = strrep(difference_eq, ' + -', ' - ');
233
234     % DC Gain calculation
235     dc_val = dcgain(D_s);
236     if isinf(dc_val) || isnan(dc_val) || abs(dc_val) < 1e-6
237         K = 1.0;
238     else
239         K = 1.0 / dc_val;
240     end
241
242     % Display results
243     fprintf('\n===== DISCRETIZATION =====\n');
244     fprintf('Continuous Controller:\n');
245     D_s
246     fprintf('Method: %s, Ts = %.4f s (%.0f Hz)\n', method, Ts, 1/Ts);
247     fprintf('\nDiscrete Transfer Function:\n');
248     D_z
249     fprintf('\nDIFFERENCE EQUATION:\n');
250     fprintf('%s\n', difference_eq);
251     fprintf('\nCOEFFICIENTS FOR ARDUINO:\n');
252     fprintf('float b0 = %.6f, b1 = %.6f, b2 = %.6f;\n', B(1), B(2), B(3)
253         );
254     fprintf('float a1 = %.6f, a2 = %.6f;\n', -A(1), -A(2));
255     fprintf('=====');
256 end

```

Appendix B: Complete Arduino Code

Listing 6: Complete Arduino firmware for closed-loop motor control

```
1  /*
2  *  Mechatronics Lab Session 7: Closed-Loop Motor Control
3  *  University of Tehran - Fall 2025
4  *
5  *  Three-Controller Implementation for DC Motor Speed Control
6  *  Uses Timer1 ISR at 200 Hz for discrete control law
7  *  H-bridge bidirectional control via L298N
8  *
9  *  To switch controllers: Change coefficients (b0, b1, b2, a1, a2)
10 */
11
12 #include <Arduino.h>
13 #include <TimerOne.h>
14
15 // ===== USER SETTINGS =====
16 const float TARGET_RPM = 800.0; // Desired speed setpoint
17
18 // CONTROLLER COEFFICIENTS - Paste from MATLAB
19 // Uncomment ONE set at a time:
20
21 // Controller 1: Moderate damping (zeta=0.4, wn=33)
22 float b0 = 0.278213, b1 = -0.480337, b2 = 0.207326;
23 float a1 = -1.986843, a2 = 0.986887;
24
25 // Controller 2: High damping (zeta=0.75, wn=42) - RECOMMENDED
26 // float b0 = 0.584569, b1 = -1.032467, b2 = 0.455886;
27 // float a1 = -1.968746, a2 = 0.968990;
28
29 // Controller 3: High speed (zeta=0.24, wn=99) - UNSTABLE AT 200 Hz
30 // float b0 = 0.942001, b1 = -1.249796, b2 = 0.414541;
31 // float a1 = -1.976380, a2 = 0.976520;
32 // =====
33
34 // ===== PIN DEFINITIONS =====
35 const uint8_t ENC_A = 2; // Encoder Channel A (INT0)
36 const uint8_t ENC_B = 4; // Encoder Channel B
37 const uint8_t ENA = 5; // H-Bridge Enable (PWM) - Timer0
38 const uint8_t IN1 = 7; // H-Bridge Direction 1
39 const uint8_t IN2 = 8; // H-Bridge Direction 2
40
41 // ===== SYSTEM CONSTANTS =====
42 const float TICKS_PER_REV = 100.0f; // Encoder resolution
43 const uint16_t FS_HZ = 200; // Sampling frequency (Hz)
44 const uint32_t Ts_us = 1000000UL / FS_HZ; // Sampling period (us)
45 const uint16_t N_SAMPLES = 400; // Data points (2 seconds)
46
47 // ===== STATE VARIABLES =====
48 // Controller state
49 volatile float currentRPM = 0.0;
50 volatile float e_k1 = 0, e_k2 = 0; // Error history
51 volatile float u_k1 = 0, u_k2 = 0; // Control output history
52
53 // Encoder state
```

```

54 volatile long ticks = 0;
55 volatile long ticks_prev = 0;
56
57 // Data logging
58 volatile float logActual[N_SAMPLES];
59 volatile uint16_t sampleIdx = 0;
60 volatile bool captureDone = false;
61
62 // ===== INTERRUPT SERVICE ROUTINES =====
63
64 /**
65  * Encoder ISR: Called on rising edge of Channel A
66  * Reads Channel B to determine rotation direction
67  */
68 void isrEncA() {
69     if (digitalRead(ENC_B)) {
70         ticks++; // Clockwise (CW) rotation
71     } else {
72         ticks--; // Counter-clockwise (CCW) rotation
73     }
74 }
75
76 /**
77  * Control ISR: Timer1 interrupt at 200 Hz
78  * Implements discrete controller difference equation
79  */
80 void controlISR() {
81     if (captureDone) return;
82
83     // ===== 1. SPEED MEASUREMENT =====
84     // Calculate RPM from encoder tick difference
85     long t = ticks;
86     long dt = t - ticks_prev;
87     ticks_prev = t;
88     currentRPM = ((float)dt / TICKS_PER_REV) * (60.0f * (float)FS_HZ);
89
90     // ===== 2. ERROR CALCULATION =====
91     float e_k = TARGET_RPM - currentRPM;
92
93     // ===== 3. DISCRETE CONTROL LAW =====
94     // Implement difference equation:
95     //  $u[k] = b_0 * e[k] + b_1 * e[k-1] + b_2 * e[k-2] - a_1 * u[k-1] - a_2 * u[k-2]$ 
96     float u_k = (b0 * e_k) + (b1 * e_k1) + (b2 * e_k2)
97         - (a1 * u_k1) - (a2 * u_k2);
98
99     // ===== 4. ANTI-WINDUP SATURATION =====
100    // Clamp control effort to PWM limits
101    if (u_k > 255.0f) u_k = 255.0f;
102    if (u_k < -255.0f) u_k = -255.0f;
103
104    // ===== 5. UPDATE STATE HISTORY =====
105    // Shift history buffers for next iteration
106    e_k2 = e_k1;
107    e_k1 = e_k;
108    u_k2 = u_k1;
109    u_k1 = u_k;
110
111    // ===== 6. H-BRIDGE BIDIRECTIONAL CONTROL =====

```

```

112 // Decompose signed control effort into direction + magnitude
113 int pwmOut = (int)u_k;
114 if (pwmOut >= 0) {
115     // Positive control: Forward rotation
116     digitalWrite(IN1, HIGH);
117     digitalWrite(IN2, LOW);
118 } else {
119     // Negative control: Reverse rotation (braking)
120     digitalWrite(IN1, LOW);
121     digitalWrite(IN2, HIGH);
122     pwmOut = -pwmOut; // Magnitude only for PWM
123 }
124 analogWrite(ENA, pwmOut);
125
126 // ===== 7. DATA LOGGING =====
127 if (sampleIdx < N_SAMPLES) {
128     logActual[sampleIdx] = currentRPM;
129     sampleIdx++;
130 } else {
131     // Experiment complete - stop motor
132     captureDone = true;
133     analogWrite(ENA, 0);
134 }
135 }
136
137 // ===== MAIN PROGRAM =====
138
139 void setup() {
140     // Initialize serial communication
141     Serial.begin(115200);
142     while(!Serial) {
143         ; // Wait for serial port connection
144     }
145
146     // Configure encoder pins with pull-ups
147     pinMode(ENC_A, INPUT_PULLUP);
148     pinMode(ENC_B, INPUT_PULLUP);
149     attachInterrupt(digitalPinToInterrupt(ENC_A), isrEncA, RISING);
150
151     // Configure motor driver pins
152     pinMode(ENA, OUTPUT);
153     pinMode(IN1, OUTPUT);
154     pinMode(IN2, OUTPUT);
155
156     // Initialize motor in stopped state
157     digitalWrite(IN1, HIGH);
158     digitalWrite(IN2, LOW);
159     analogWrite(ENA, 0);
160
161     // Display startup information
162     Serial.println("\n=====");
163     Serial.println("Mechatronics Lab 7: Closed-Loop Control");
164     Serial.println("=====");
165     Serial.print("Target RPM: ");
166     Serial.println(TARGET_RPM);
167     Serial.print("Sampling Rate: ");
168     Serial.print(FS_HZ);
169     Serial.println(" Hz");

```

```

170 Serial.print("Duration: ");
171 Serial.print((float)N_SAMPLES / FS_HZ);
172 Serial.println(" seconds");
173 Serial.println("=====\n");
174 Serial.println("Starting experiment in 2 seconds...");
175 delay(2000);
176
177 // Initialize Timer1 for deterministic control loop
178 Timer1.initialize(Ts_us);
179 Timer1.attachInterrupt(controlISR);
180
181 Serial.println("Controller active - collecting data...\n");
182 }
183
184 void loop() {
185     // Wait for data collection to complete
186     if (captureDone) {
187         // Disable control loop
188         Timer1.detachInterrupt();
189
190         // Transmit collected data via serial
191         Serial.println("== EXPERIMENTAL DATA START ==");
192         for (uint16_t i = 0; i < N_SAMPLES; i++) {
193             Serial.println(logActual[i], 2); // 2 decimal places
194         }
195         Serial.println("== EXPERIMENTAL DATA END ==");
196
197         // Calculate statistics
198         float sum = 0;
199         for (uint16_t i = N_SAMPLES - 20; i < N_SAMPLES; i++) {
200             sum += logActual[i];
201         }
202         float finalRPM = sum / 20.0;
203         float error = ((TARGET_RPM - finalRPM) / TARGET_RPM) * 100.0;
204
205         Serial.println("\n== PERFORMANCE METRICS ==");
206         Serial.print("Target RPM: ");
207         Serial.println(TARGET_RPM);
208         Serial.print("Final RPM (avg last 20): ");
209         Serial.println(finalRPM, 2);
210         Serial.print("Steady-State Error: ");
211         Serial.print(error, 2);
212         Serial.println("%");
213         Serial.println("=====\n");
214
215         Serial.println("Experiment complete. Reset Arduino to run again.");
216
217         // Halt - infinite loop
218         while(1) {
219             ; // Do nothing
220         }
221     }
222 }

```

Appendix C: MATLAB Data Acquisition Script

Listing 7: MATLAB script for serial data acquisition and plotting

```
1 %% Motor Step Response Data Acquisition
2 % Reads data from Arduino after closed-loop experiment
3 % Plots results and calculates performance metrics
4
5 clc; close all;
6 if ~isempty(instrfind), fclose(instrfind); delete(instrfind); end
7
8 % ===== SETTINGS =====
9 serialPort = "COM15"; % <--- CHANGE THIS TO YOUR PORT
10 baudRate = 115200;
11 fs = 200; % Sampling frequency (Hz)
12 duration = 2; % Duration (seconds)
13 numSamples = fs * duration;
14
15 % ===== CONNECT TO ARDUINO =====
16 fprintf('1. Opening serial port %s...\n', serialPort);
17 % Note: Opening port triggers DTR reset on Arduino
18 device = serialport(serialPort, baudRate);
19 configureTerminator(device, "CR/LF");
20 flush(device);
21
22 fprintf('2. Arduino is resetting and starting experiment...\n');
23 fprintf(' (Wait %.1f seconds for data collection)\n', duration + 1);
24 pause(duration + 1);
25
26 % ===== READ DATA =====
27 fprintf('3. Reading experimental data...\n');
28 rpmData = zeros(numSamples, 1);
29
30 for i = 1:numSamples
31     try
32         lineStr = readline(device);
33         val = str2double(lineStr);
34
35         if isnan(val)
36             val = 0; % Handle NaN readings
37         end
38         rpmData(i) = val;
39
40         % Progress indicator
41         if mod(i, 50) == 0
42             fprintf(' Received %d / %d samples\n', i, numSamples);
43         end
44     catch
45         warning('Timeout waiting for data at sample %d', i);
46         break;
47     end
48 end
49
50 fprintf(' Data acquisition complete: %d samples received.\n', length(
    rpmData));
51 clear device; % Close serial port
52
```



```

53 % ===== ANALYZE DATA =====
54 t = (0:numSamples-1) / fs;
55
56 % Calculate metrics
57 final_speed = mean(rpmData(end-20:end));
58 max_speed = max(rpmData);
59 min_speed = min(rpmData);
60
61 % Find settling time (2% criterion)
62 settlingBand = 0.02 * final_speed;
63 upperBound = final_speed + settlingBand;
64 lowerBound = final_speed - settlingBand;
65
66 settlingIdx = numSamples;
67 for i = numSamples:-1:1
68     if rpmData(i) > upperBound || rpmData(i) < lowerBound
69         settlingIdx = i;
70         break;
71     end
72 end
73 settlingTime = settlingIdx / fs;
74
75 % Calculate overshoot
76 if max_speed > final_speed
77     overshoot = ((max_speed - final_speed) / final_speed) * 100;
78 else
79     overshoot = 0;
80 end
81
82 % ===== PLOT RESULTS =====
83 figure('Color', 'w', 'Position', [100, 100, 900, 600]);
84
85 % Main time-series plot
86 plot(t, rpmData, 'b-', 'LineWidth', 1.5);
87 hold on;
88 yline(final_speed, 'r--', 'LineWidth', 1.5);
89 yline(upperBound, 'k:', 'LineWidth', 1);
90 yline(lowerBound, 'k:', 'LineWidth', 1);
91 xline(settlingTime, 'g--', 'LineWidth', 1.5);
92
93 grid on;
94 xlabel('Time (s)', 'FontSize', 12);
95 ylabel('Speed (RPM)', 'FontSize', 12);
96 title('Closed-Loop Step Response', 'FontSize', 14, 'FontWeight', 'bold')
97 ;
98 xlim([0 duration]);
99
100 % Legend
101 legend({'Measured Speed', ...
102         sprintf('Final Value: %.1f RPM', final_speed), ...
103         'Settling Band ( 2 %)', ...
104         '', ...
105         sprintf('Settling Time: %.3f s', settlingTime)}, ...
106        'Location', 'southeast');
107
108 % Add text box with metrics
109 textStr = sprintf(['Performance Metrics:\n' ...
110                    'Final Speed: %.1f RPM\n' ...

```

```

110         'Max Speed: %.1f RPM\n' ...
111         'Overshoot: %.1f%%\n' ...
112         'Settling Time: %.3f s'], ...
113         final_speed, max_speed, overshoot, settlingTime);
114 annotation('textbox', [0.15, 0.65, 0.3, 0.2], ...
115         'String', textStr, ...
116         'FitBoxToText', 'on', ...
117         'BackgroundColor', 'white', ...
118         'EdgeColor', 'black', ...
119         'FontSize', 10);
120
121 % ===== SAVE DATA =====
122 % Save to workspace for further analysis
123 fprintf('\n=== PERFORMANCE SUMMARY ===\n');
124 fprintf('Final Speed: %.2f RPM\n', final_speed);
125 fprintf('Maximum Speed: %.2f RPM\n', max_speed);
126 fprintf('Minimum Speed: %.2f RPM\n', min_speed);
127 fprintf('Overshoot: %.2f%%\n', overshoot);
128 fprintf('Settling Time: %.3f s\n', settlingTime);
129 fprintf('=====\n');
130
131 % Prompt to save
132 saveName = input('Enter filename to save data (or press Enter to skip):
133                 ', 's');
134 if ~isempty(saveName)
135     save(saveName, 'rpmData', 't', 'final_speed', 'overshoot', '
136         settlingTime', 'fs');
137     fprintf('Data saved to %s.mat\n', saveName);
138 end

```